Untying Prefill and Decode Loops in Looped Language Models

Yang Hu

Abstract

Looped language models run a block of layers several times per token and use the same number of loops to read the prompt (prefill) and to generate each token (decode). Chain-of-thought generation, however, is almost all decoding. On Ouro-2.6B at batch 1, prefill takes 0.4–2.8% of the time of a chain-of-thought answer, so extra prefill loops cost little. We untie the two loop counts and read the prompt with more loops than we use to decode each token. The released model cannot use the states of a longer prefill, so we distill a small adapter (2.4% of the parameters): LoRA on the decoder for a fixed prefill depth, or LoopBridge, whose per-loop key and value maps keep retrieval over 16K-token prompts when one adapter serves every trained pair of depths. On Ouro-2.6B with one decode loop, four prefill loops raise the average over 11 benchmarks from 44.2 to 57.1 (56.8 with LoopBridge), above the original model distilled to one loop on the same data (49.8), and in the same measured time they are 14–15 points more accurate than the original model. The gain peaks after two to five prefill loops and holds on Ouro-1.4B, the Thinking models and looped decision models. Code, results and per-item outputs: `https://github.com/yanghu819/untying-prefill-decode`.

1 Introduction

Looped language models such as Ouro (Dehghani et al., 2019; Zhu et al., 2025) reuse a block of layers several times for each token. Ouro-2.6B runs its block four times, for every prompt token and for every answer token (Figure 1a). The two phases cost very different amounts of time. Prefill reads all prompt tokens in one parallel pass, while decoding repeats for every generated token, so on chain-of-thought tasks decoding takes over 97% of the time (Figure 2).

This asymmetry suggests untying the two loop counts (Figure 1b): read the prompt with m loops, which is nearly free, and decode each answer token with fewer loops, $n < m$, which saves time on every token. Deep encoders with shallow decoders make a similar trade across two networks in machine translation (Kasai et al., 2021); here both phases share one looped model. The released model cannot make this trade by itself. Prefilled with four loops and decoded with one, Ouro-2.6B averages 26.6 points over 11 benchmarks, far below the 44.2 it reaches with one loop for both (Table 1), because its decoder has never read the states of a longer prefill.

We close this gap with a small adapter distilled from the original four-loop model (Section 3). When the prefill depth is fixed, LoRA on the de-

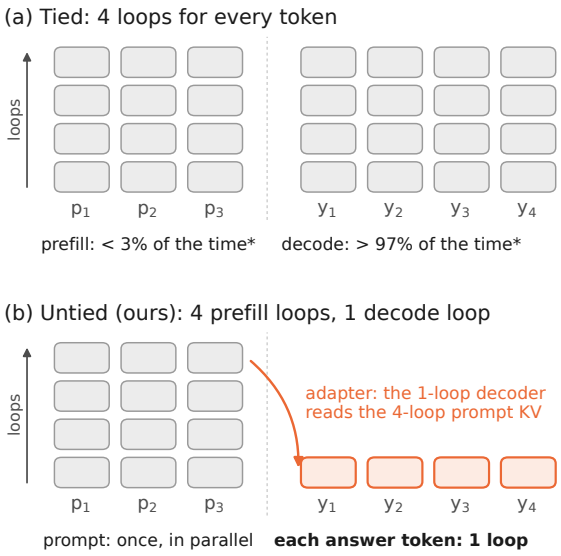


Figure 1: **Untying prefill and decode loops.** (a) A looped model such as Ouro runs every token, prompt and answer, through the same loops, although the prompt is read once in parallel and the answer is written token by token. (b) We read the prompt with more loops ($m = 4$) and decode each answer token with fewer ($n = 1$); a small adapter lets the short decoder read the long prefill's key-value cache. *Chain-of-thought answers at batch 1 (Figure 2).

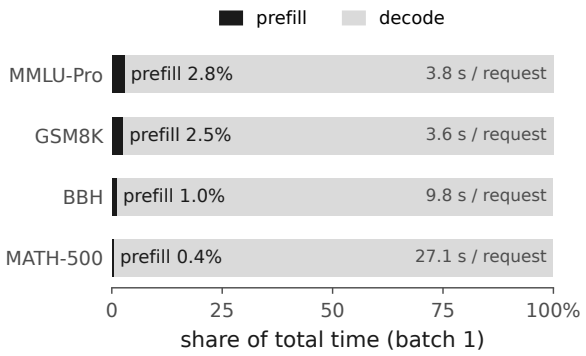


Figure 2: **Prefill is a sliver of the time.** Share of the total time over 100 requests per task spent reading the prompt (prefill, black) and writing the answer (decode, grey): the released Ouro-2.6B at one loop (Original 1/1), batch 1, one B300. Per-request medians and every configuration: Appendix A.

coder is enough. One adapter can also serve every trained pair of prefill and decode depths; LoopBridge adds a key map and a value map on the prompt cache for every decode loop, which keep retrieval over 16K-token prompts when it does. The maps fold into the prompt cache, so decoding runs the original model's attention at its cost per token. Our main findings on Ouro-2.6B:

- **Same decode loops, more accuracy.** With one decode loop, four prefill loops raise the 11-benchmark average from 44.2 to 57.1 (56.8 with LoopBridge), above the original model distilled to one loop on the same data (49.8; 50.2 with as many parameters as the adapter); with two decode loops, from 63.0 to 67.3 and 67.2 (Table 1).
- **Same time, more accuracy.** At one decode loop and about 11 seconds per request, untying is 14–15 points more accurate than the original model (Figure 6); served in batches of 32, it keeps 14.5 points for 15% more time per request.
- **A few prefill loops suffice.** The gain peaks after two to five prefill loops (Figure 5).
- **It transfers.** The pattern holds on Ouro-1.4B, on looped decision models and on Ouro-2.6B-Thinking, where four prefill loops and two decode loops come within 1.3 points of the original four-loop model on average and, served in batches, match it on GSM8K in about half its time (Section 5.7).

2 Related Work

Looped models. Looped and recursive transformers (Dehghani et al., 2019; Geiping et al., 2025; Zhu et al., 2025; Bae et al., 2025a,b) reuse a block of layers to add depth without parameters, and looped models can simulate chain-of-thought steps in latent space (Saunshi et al., 2025). Relaxed Recursive Transformers (Bae et al., 2025a) convert pretrained models into looped ones with per-loop LoRA; Ouro, ELR and Dory reuse one loop's KV cache (Zhu et al., 2025; Ladhak et al., 2026; Fishman et al., 2026). All of them run the same number of loops for the prompt and for every answer token. We give the two phases different loop counts within one model, and train a small adapter that makes this work after pretraining.

Depth per phase and per token. Early exit (Schuster et al., 2022; Elhoushi et al., 2024) and Continuous Depth Batching (Schwethelm et al., 2026) vary the depth per token; CALM must also fill the skipped layers' KV for later tokens, the problem our maps solve per loop. LoopDuet (concurrent; Du et al., 2026) sets the two loop budgets of Huginn without training; Huginn was trained with random loop counts, whereas fixed-depth Ouro fails without training (Table 1). Deep encoders with shallow decoders (Kasai et al., 2021; Bergner et al., 2024; S et al., 2024) and BridgeKV (concurrent; Behnam et al., 2026) put more depth on the prompt across separate networks; SwiftKV (Qiao et al., 2025) puts less. Serving systems already run prefill and decode on separate hardware (Zhong et al., 2024; Patel et al., 2024); untying the loops lets each phase also use its own depth.

Asymmetric KV producers and consumers. YOCO (Sun et al., 2024) has a self-decoder produce one global KV cache that a cross-decoder reads, and KITE (Hu et al., 2026) expands a checkpoint into two towers so that bulk prefill runs only the KV-producing tower. These architectures are built for the split; we adapt an existing looped model after pretraining and choose m and n at inference. More related work is in Appendix F.

3 Method

3.1 Prefill and decode in a looped model

A looped model applies one block of layers n times per token, and every loop of every layer writes its own keys and values. We write m/n for

a model that prefills the prompt with m loops and decodes each answer token with n loops; the released model runs *Original* n/n. Prefill runs once over the whole prompt in parallel, decoding once per answer token. If prefill takes a share p of the time at one loop, it takes about $m\,p$ at m loops, so when p is a few percent, extra prefill loops cost little (Section 5.1).

3.2 Untying the loops

With $m > n$, decode loop k reads the prompt cache of prefill loop $\max(m - n + k, 1)$. The last decode loop thus reads the last prefill loop, and at $m = n$ every decode loop reads exactly the cache the original model would read. The frozen model fails at this (the training-free rows of Table 1), because its decoder has only read prompt states written by the same loops. A small adapter fixes it.

Fixed prefill depth. For one prefill depth m, we train rank-16 LoRA (Hu et al., 2022) on the query, key, value and output projections, active only while decoding, on the pairs $m/1, \ldots, m/4$. We call this *Ours, fixed* m.

Any prefill depth: LoopBridge. To serve every (m, n) with one set of weights without losing retrieval over long prompts, LoopBridge (Figure 3; pseudocode in Appendix E) adds, for each decode loop, layer and KV head, a key map and a value map on the prompt cache that loop reads. The key map is one complex scale per rotary frequency pair; it commutes with RoPE (Su et al., 2024), so it can be applied to the query instead of the cache. The value map is linear and is applied to the attention output, since attention is linear in the values. Prompt and answer attention share one softmax, merged exactly through their log-sum-exps. Both maps can also be folded into the prompt cache once at prefill, one mapped copy per decode loop (as many copies as the original model stores at n loops). Decoding then runs the original model's attention at its cost per token; we time this form. A lighter variant keeps a single prompt cache that every decode loop reads and saves memory (Table 2a).

Training. We distil (Hinton et al., 2015) the adapter from Original 4/4 with a KL loss on the answer tokens of 48M tokens of reasoning traces, instruction data, long-context data and web text, sampling one (m, n) pair per micro-batch. The adapter has 63M parameters, 2.4% of Ouro-2.6B. The main LoopBridge is trained on $m, n \leq 4$, a second one on $m \in \{1, 2, 3, 4, 6, 8, 12, 16\}$ and $n \leq 4$ for the depth sweep (Appendix G).

4 Experimental Setup

Models and benchmarks. Ouro-2.6B and Ouro-1.4B and their Thinking variants (Zhu et al., 2025) (four loops), and Lev decision models (Song et al., 2026) (Section 5.7). The base models are evaluated on 11 benchmarks with the Ouro protocol: MMLU, MMLU-Pro, BBH, ARC-Challenge, HellaSwag, Winogrande, GSM8K, MATH-500, HumanEval+, MBPP+ and RULER-16K (Appendix H); the Thinking models on eight benchmarks including HELMET (Yen et al., 2025).

Baselines. *Original* n/n is the released model with n loops. *Training-free* $4/n$ unties the loops without an adapter. *Distilled* n/n trains LoRA on the n-loop model to match Original 4/4 with the adapter's data, teacher and 48M tokens, at rank 16 and at rank 80 (62.9M parameters, as many as the adapter); scaling its learning rate by 0.3 or 3 changes its average by at most 0.6 points (Table 2c).

Time. We time each request at batch 1 on one B300 GPU in bfloat16 with greedy decoding, split into prefill (to the first token) and decode, and score the replayed answers with the evaluation's rule, so accuracy and time come from the same requests. On the base model we time 100 requests per task on the four tasks that write chain-of-thought answers (GSM8K, MATH-500, BBH, MMLU-Pro), the same for every configuration, and average two passes in opposite orders on an otherwise idle machine (Appendix A). Comparisons are paired over items with 95% bootstrap intervals.

5 Results

5.1 Where the time goes

Figure 2 splits the time of each request into prefill and decode. Chain-of-thought tasks read a short prompt and write a long answer, so prefill takes 0.4% (MATH-500) to 2.8% (MMLU-Pro) of the time of the original model at one loop, and 0.3–2.8% for Original 1/1, 2/2 and 4/4 and our adapters at one and two decode loops. Reasoning models write longer answers: on 192 GSM8K problems Ouro-2.6B-Thinking reads about 100 tokens and writes about 900, and prefill takes 0.2–0.3% of the

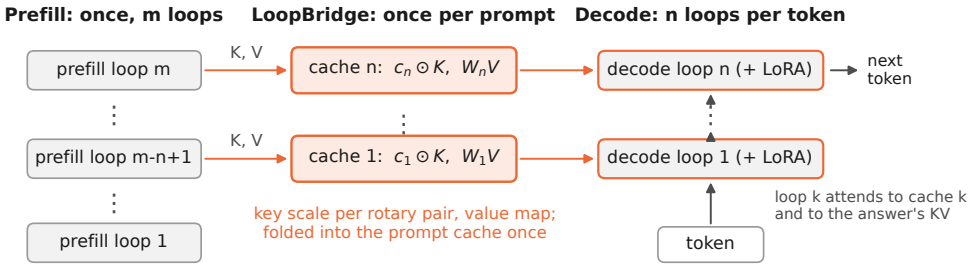


Figure 3: **LoopBridge, as timed.** The frozen model prefills the prompt once with m loops. Once per prompt, LoopBridge maps the keys and values of prefill loop $\max(m-n+k, 1)$ into a cache for decode loop k: a scale c_k on each rotary pair of the keys and a linear map W_k of the values. Every answer token then runs n loops of the frozen model with LoRA, and loop k attends to cache k and to the answer's own keys and values with the original attention. Applying the maps at every step instead (to the query and the attention output) is equivalent in exact arithmetic; pseudocode: Figure 13 in Appendix E.

time; on MATH-500, 0.1%. Only long prompts with short answers shift the balance (HELMET: 27–30%; Appendix A).

5.2 Main results

Table 1 compares the two adapters with the original and distilled models at the same number of decode loops. Without an adapter, untying fails (26.6 and 30.9). With one decode loop, four prefill loops raise the average from 44.2 for Original 1/1 to 57.1 with the fixed-depth adapter and 56.8 with LoopBridge, about 7 points above the original model distilled to one loop on the same data (49.8), also at rank 80 (50.2). Both adapters beat Original 1/1 on ten of the 11 benchmarks, by 15 to 24 points on MMLU, MMLU-Pro, BBH, ARC-Challenge, HumanEval+ and RULER-16K; on MATH-500 the four-shot exemplars favour one prefill loop, and without them four prefill loops gain 8.0 points (Appendix N). With two decode loops the two adapters average 67.3 and 67.2, against 63.0 for Original 2/2 and 65.3 for Distilled 2/2, within three points of Original 4/4 with half its decode loops. The ranking holds over three training seeds, at other learning rates for both methods (Table 2c), and without the benchmark items that overlap the training data (Appendix G).

5.3 Fewer decode loops, more prefill loops

Figure 4 varies the decode loops. The original model loses most of its accuracy when it decodes with fewer loops; with a four-loop prefill the decoder keeps much of it. At one decode loop, MMLU rises from 51.5 to 67.1, BBH from 30.9 to 52.7 and HumanEval+ from 44.5 to 65.2. Decode loops matter most for multi-step math (Original GSM8K: 32.4 at one loop, 69.4 at four), and four prefill loops add 13 points there. With three or four decode loops LoopBridge tracks the original model, also on long-context retrieval (RULER-16K at 4/3: 79.8 against 78.7 for Original 3/3).

5.4 How many prefill loops?

We now fix the decode loops and vary only the prefill loops, with one set of weights. At one decode loop, a few more prefill loops raise GSM8K by 10.2 points, MMLU-Pro by 24.1, HumanEval+ by 17.7 and RULER-16K by 10.1 (Figure 5). With the main LoopBridge and one decode loop, four prefill loops instead of one raise the 11-benchmark average from 47.2 to 56.8 on the same weights (a gain of 9.0 ± 0.6 over three training seeds): the gain comes from the prefill loops, not only from training. Ouro-2.6B-Thinking behaves the same way (GSM8K 90.7 to 94.1 and MMLU-Pro 36.4 to 51.0 from $m = 1$ to $m = 4$ at two decode loops).

The gain peaks after a few loops. To place its end, we pick, on one half of each benchmark, the first depth that none of the next three tested depths beats by more than 0.5 points, and report accuracy on the other half (Appendix B). For the adapter trained up to 16, this stop is at two to five prefill loops in 14 of the 18 benchmark and decode-depth settings (two or three in 11). It stays there over three training seeds and when the prefill loops are trained as well (Appendix B), and Ouro-1.4B peaks at three or four prefill loops in 13 of 18 settings (Appendix C). For a new workload, the same rule on a few hundred held-out requests picks the depth; without them, $m = 4$, the depth the backbone was trained with, is a good default (Table 1).

method	m/n	MMLU	MMLU-Pro	BBH	ARC-C	HellaSw.	Winogr.	GSM8K	MATH	HE+	MBPP+	RULER	Avg.
Original	1/1	51.5	26.9	30.9	47.7	69.0	61.1	32.4	32.2	44.5	49.2	41.4	44.2
Training-free	4/1	61.9	0.0	0.8	37.5	40.4	51.9	8.9	4.6	29.3	48.9	8.0	26.6
Distilled	1/1	52.7	26.7	40.4	55.0	70.1	66.5	40.4	**37.0**	54.3	50.3	54.7	49.8
Distilled, rank 80	1/1	53.4	27.9	40.5	55.3	70.2	66.6	40.3	36.4	55.5	50.0	56.2	50.2
Ours, fixed m	4/1	**68.1**	**47.4**	**54.4**	**62.9**	**72.7**	67.5	**47.2**	25.0	**67.7**	**57.4**	58.3	**57.1**
Ours, LoopBridge	4/1	67.1	45.0	52.7	62.8	72.7	**68.0**	45.0	25.4	65.2	56.4	**64.7**	56.8
Original	2/2	67.7	45.0	66.6	62.2	77.8	71.9	62.9	**50.6**	66.5	61.1	60.8	63.0
Training-free	4/2	71.3	4.0	4.0	41.0	50.6	65.5	11.7	0.0	7.3	33.1	52.0	30.9
Distilled	2/2	67.2	46.2	70.5	62.9	77.6	74.6	69.6	46.8	69.5	63.0	70.3	65.3
Distilled, rank 80	2/2	66.9	44.8	70.5	62.6	77.4	75.2	**70.1**	48.2	69.5	62.7	71.8	65.4
Ours, fixed m	4/2	73.2	**54.0**	72.8	**65.4**	**78.2**	**75.3**	67.7	47.2	68.9	63.0	**74.5**	**67.3**
Ours, LoopBridge	4/2	**73.5**	**54.0**	**73.5**	65.0	**78.2**	75.1	66.3	45.8	**70.1**	**64.3**	73.5	67.2
Original	4/4	74.5	55.2	80.4	66.5	79.5	77.7	69.4	53.6	69.5	64.5	81.3	70.2

Table 1: **Untying beats the original and distilled models at the same decode loops.** Ouro-2.6B, accuracy (%) grouped by decode loops n; ours shaded, bold: best in each group. Original: the released model. Training-free: four prefill loops without an adapter. Distilled: LoRA on the n-loop model distilled from Original 4/4 with the adapters' data, teacher and tokens. MATH: MATH-500 (4-shot); HE+: HumanEval+; RULER: RULER at 16K tokens. Each column on the items all rows share. Over three training seeds (mean $\pm$ s.d.), the fixed-depth adapter averages 56.8 ± 0.3 at 4/1 and 67.3 ± 0.4 at 4/2, LoopBridge 56.7 ± 0.1 and 67.1 ± 0.2, Distilled 49.7 ± 0.3 at 1/1 and 65.1 ± 0.2 at 2/2 (rank 80: 50.2 over two seeds and 65.3 ± 0.1).

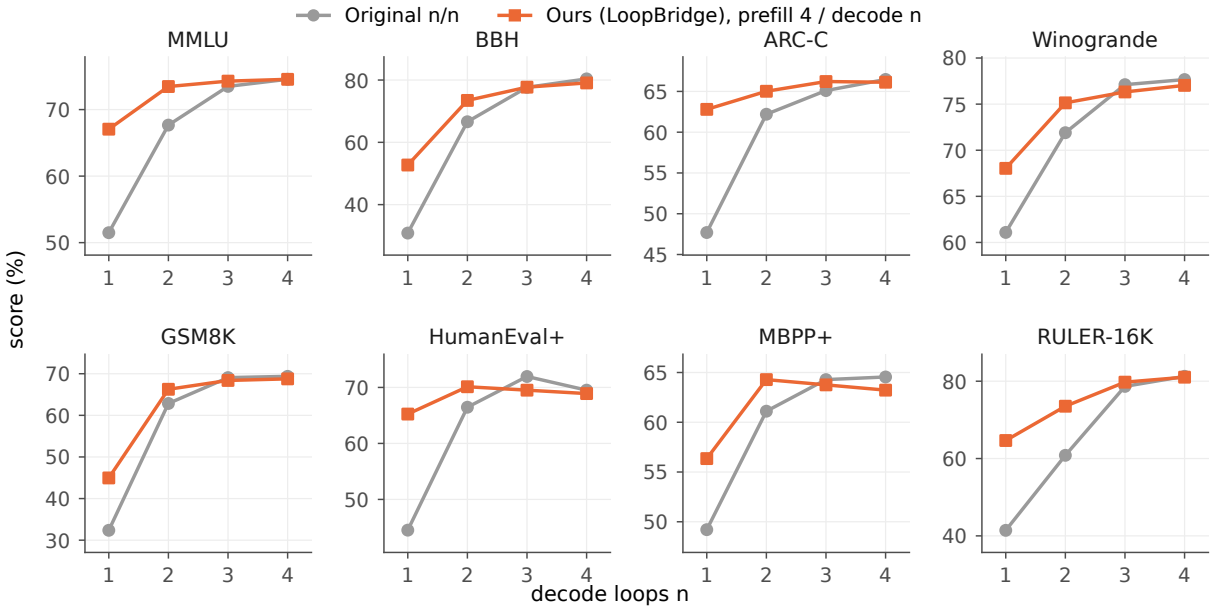


Figure 4: **Four prefill loops recover much of the accuracy lost by decoding with fewer loops.** Ouro-2.6B, accuracy (%) against the number of decode loops n. Grey: the original model (n/n). Orange: LoopBridge with four prefill loops ($4/n$). Eight of the 11 benchmarks; each panel uses the items all points share; every benchmark and (m, n) cell, including MATH-500: Table 8.

The extra loops are cheap: with folded maps, time per request grows by 16% from $m = 1$ to $m = 4$ at one decode loop, almost all because answers get longer (276 to 319 output tokens on average).

5.5 Same measured time

Equal decode loops do not mean equal time, so we measure it on the same requests (Figures 6 and 7). With the maps folded into the prompt cache, LoopBridge decodes within 10% of the original model's time per token. At one decode loop and about 11 seconds per request, the fixed-depth adapter scores 42.0 and LoopBridge 41.2 on the four chain-of-thought tasks, against 27.0 for Original 1/1 and 34.2 for Distilled 1/1; paired over the 400 requests, LoopBridge is $+14.2$ [9.0, 19.8] points above Original 1/1 and $+7.0$ [1.8, 12.5] above Distilled 1/1, and the fixed-depth adapter is $+15.0$ [9.8, 20.5] and

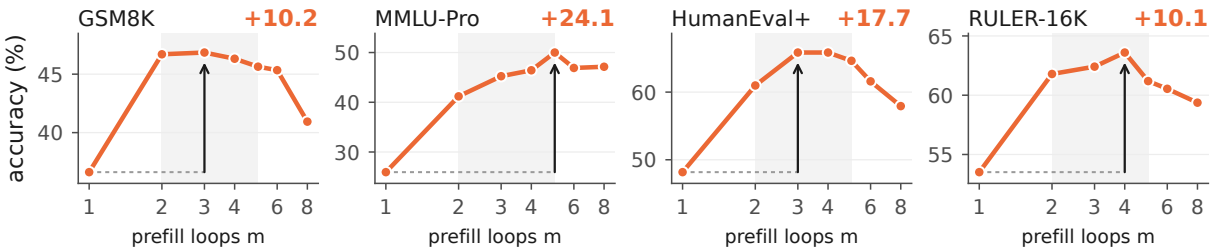


Figure 5: **A few more prefill loops buy a lot of accuracy.** One decode loop, Ouro-2.6B with a LoopBridge trained on prefill depths 1 to 16 (one set of weights for every m, no depth extrapolated). Orange: gain from one prefill loop to the best observed depth; shaded: two to five loops, where that depth lies for all four tasks (a range, not an interval). m up to 8 shown; depths 12 and 16 for the timed tasks (GSM8K, MMLU-Pro, MATH-500, BBH), two decode loops, the main adapter and time per request: Figure 10; stopping depths by a fixed rule for every benchmark: Table 6.

prefill 4 / decode 1	all (m, n)	average	RULER-16K
Ours, fixed m=4: LoRA r16	×	57.1	58.3
LoRA r16, all (m, n)	✓	55.0	49.8
LoRA r80, all (m, n)	✓	56.3	55.8
LoRA r16 + KV maps (LoopBridge)	✓	56.8	64.7
LoopBridge, one shared cache	✓	56.4	62.8

(a) What LoopBridge adds.

tokens	6M	12M	24M	48M
4/1	53.7	55.5	55.7	56.4
4/2	65.2	65.9	66.1	66.4

(b) Training tokens (one-cache LoopBridge).

learning rate	×0.3	×1	×3
Distilled 1/1	49.2	49.8	50.1
LoopBridge 4/1	55.5	56.8	57.4

(c) Learning-rate schedule scaled.

Table 2: **Ablations**, Ouro-2.6B, 11-benchmark average (%) unless noted; default shaded. (a) "all (m, n)": one set of weights serves every prefill and decode depth; the key/value maps restore the retrieval over 16K-token prompts (RULER-16K) that an all-pairs LoRA loses, also at rank 80 (r80 and LoopBridge both train 63M parameters). (b) LoopBridge with one shared cache trained on fewer answer tokens (rank 16). (c) The whole learning-rate schedule scaled for the distilled control and for LoopBridge.

+7.8 [2.5, 13.2]. The advantage is largest when decoding must stay at one loop, as under a bound on the time per generated token, which a mix of one- and two-loop requests does not meet: with two decode loops, untying and distillation reach about the same accuracy in about the same time (Figure 7).

Batching makes the extra prefill loops dearer, since the batch shares each decode step while every prompt is still prefilled on its own. Served with continuous batching (Figure 14 in Appendix K), the four prefill loops take 27% of the wall time at batch 32. LoopBridge 4/1 then needs 1.15× and 1.17× the time per request of Original 1/1 and Distilled 1/1 and stays +14.5 [9.0, 19.8] and +8.0 [2.8, 13.5] points above them; a second decode loop costs 1.38–1.48× its time.

Measured as decode throughput instead (Figure 12), the one-cache LoopBridge with four prefill loops is more accurate than Original n/n at one or two decode loops and faster at three or four, since one stored prompt cache admits larger batches; at 4/2 it has 1.19× and 3.2× the throughput of Original 2/2 and 4/4.

On Ouro-2.6B-Thinking, prefill 4 / decode 2 comes within 1.3 points of Original 4/4 on average at 3.2 times its decode throughput (Figure 15). Served in batches of 16 or 32, LoopBridge 4/2 matches Original 4/4 on 400 GSM8K problems (96.5 against 95.5) in 0.49–0.58 of its time and beats Original 2/2 and Distilled 2/2 by 2.2 to 4.2 points in 0.80–1.02 of their time (Table 16).

5.6 Ablations

What LoopBridge adds. A LoRA trained at one prefill depth serves that depth. Trained on all (m, n) pairs with the maps fixed at the identity, the same LoRA serves every depth but loses 1.8 points on average and 14.9 on RULER-16K at 4/1 (Table 2a), while at 4K and 8K tokens the two differ by at most 1.4 points. More capacity does not close the gap at 16K tokens: at rank

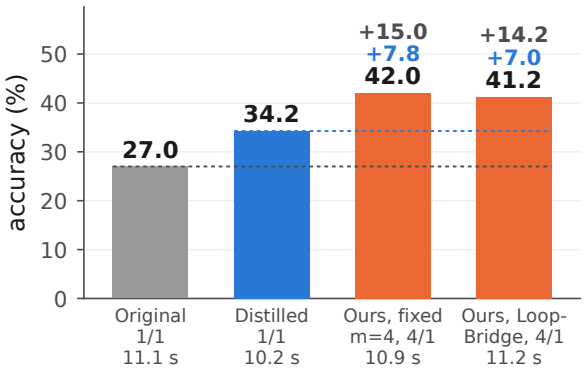


Figure 6: **Same time, much higher accuracy.** One decode loop, Ouro-2.6B: mean accuracy and mean time per request over the same 100 requests of each of GSM8K, MATH-500, BBH and MMLU-Pro (batch 1, one B300, mean of two passes). With four prefill loops, both adapters take about the time Original 1/1 needs; grey and blue numbers: gain over Original 1/1 and over Distilled 1/1 (paired intervals in Section 5.5). Every configuration: Figure 7.

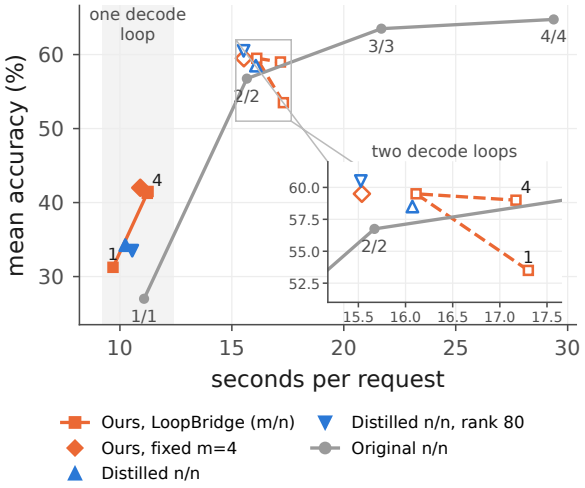


Figure 7: **Untying raises the accuracy–time frontier at low latency.** Ouro-2.6B, mean accuracy and mean time per request over the same 100 requests of each of GSM8K, MATH-500, BBH and MMLU-Pro (batch 1, one B300, mean of two passes); filled: one decode loop, hollow: two; labels: prefill loops m or n/n. At one decode loop (shaded) both adapters sit well above the original and distilled models; with two decode loops the methods reach similar accuracy in similar time.

80, with as many parameters as LoopBridge, the all-pairs LoRA is 8.8 [6.1, 11.7] points below LoopBridge on RULER-16K. Over three training seeds of each, the rank-80 LoRA matches LoopBridge on average (56.7 for both) but not on RULER-16K (57.6±1.6 against 63.9±1.0; rank 16: 44.7±4.4). The maps help where retrieval is exact, the prompt longest and the decoder shortest: with two decode loops the three all-pairs adapters reach 71.6 to 73.7 on RULER-16K, and on HELMET and LongBench they lie within three points of each other, with the rank-80 LoRA ahead on HELMET (Appendix L). One LoopBridge thus serves every depth, keeps retrieval over 16K-token prompts and matches the fixed-depth adapter on average. One shared prompt cache saves memory for 0.4 points.

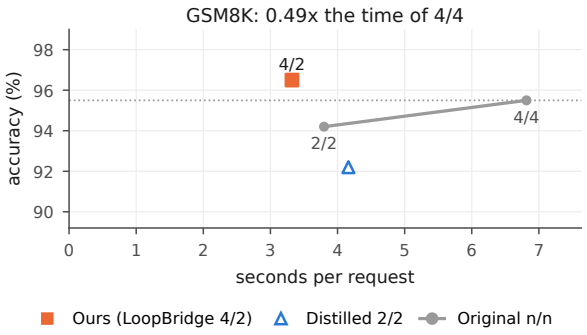


Figure 8: **The reasoning model: four-loop accuracy at two-loop cost.** Ouro-2.6B-Thinking served with continuous batching (batch 16, one B300; chat template, 4,096- and 8,192-token budgets): accuracy of the served answers against wall seconds per request on 400 GSM8K problems and all of MATH-500. Ours: LoopBridge with per-loop caches and the maps folded in, prefill 4 / decode 2; dotted: Original 4/4. Numbers: Table 16.

Data and learning rate. LoopBridge needs little data: with 6M answer tokens, an eighth of the default budget, the one-cache LoopBridge already averages 53.7 at 4/1, above the distilled model trained on 48M (49.8) (Table 2b); with 12M tokens, LoRA rank 4 or 64 instead of 16 changes its averages by at most 0.6 points. Scaling the learning-rate schedule by 0.3 or 3 moves the distilled control by at most 0.6 points and LoopBridge by at most 1.4; LoopBridge's weakest setting (55.5) is still above the best distilled one (50.1; Table 2c).

5.7 Other models

Ouro-1.4B. With its own LoopBridge, four prefill loops and one decode loop average 43.9 over the 11 benchmarks, against 34.5 for Original 1/1 and 40.0 for Original 1/1 distilled on the same data (Table 10).

Thinking models. On Ouro-2.6B-Thinking, prefill 4 / decode 2 scores higher than Original 2/2 on all eight benchmarks of Figure 11 (each on the items both share), by up to 13 points on HELMET

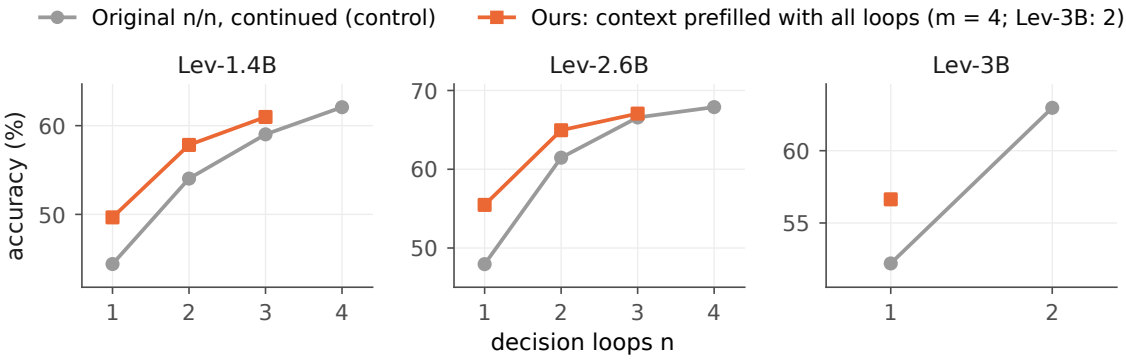


Figure 9: **Decision models gain when the context gets more loops than the decision.** Lev decision models (Song et al., 2026), LevBench development set (3,270 questions), accuracy against decision loops n. Grey: Original n/n continued with the same data and updates. Orange: the context prefilled with all of the backbone's loops ($m = 4$; Lev-3B: $m = 2$), decided with $n < m$ loops; one fixed m, no selection. All pairs: Table 20.

and 7 on MMLU-Pro and BBH; it is above Original 2/2 distilled on the same data on all eight and within two points of Original 4/4 on six of them (Figure 11, Table 12). Within one adapter, four prefill loops instead of one raise MMLU-Pro by 18.6 points on Ouro-2.6B-Thinking and by 17.1 on Ouro-1.4B-Thinking (Table 11).

Looped decision models. Lev (Song et al., 2026) turns looped language models into decision models that read a long context and score the options of a question after every loop. Prefilling the context with more loops than the decision uses beats continuing the original model on the same data on 12 of 13 (m, n) pairs, most of all with one decision loop: $+5.3$ [3.4, 7.2] on Lev-1.4B, $+7.5$ [5.7, 9.5] on Lev-2.6B and $+4.4$ [2.8, 6.1] on Lev-3B (Figure 9). The reverse, fewer prefill than decision loops, is worse on all 13 pairs (Appendix M): the context gains from many loops, and the decision can then stop early.

6 Conclusion

Chain-of-thought generation in looped models is almost all decoding, so the prompt can be read with more loops than each answer token. A small distilled adapter makes this untying work: with four prefill loops, one decode loop gains about 13 points on average over the original model and 14–15 points in the same measured time, and LoopBridge keeps retrieval over 16K-token prompts when one adapter serves every trained pair of depths. Reading the prompt with many loops and decoding with few is a cheap way to spend test-time compute in looped models.

Limitations

Time is measured on one GPU type, at batch 1 except for the batched serving check; at batch 32 four prefill loops already take 27% of the wall time, and with larger batches or faster decoding their share grows further. Applied at every decode step, the maps and a second attention call cost 19–40% more decode time per generated token; folding the maps into the prompt cache removes this cost but stores one prompt cache per decode loop, as the original model does at n loops. The depth sweep uses the per-step form. With two decode loops, untying and distillation reach about the same accuracy in the same time, and on long chain-of-thought a second decode loop buys more than more prefill loops (Figure 7). The prefill depth is fixed per setting, not routed per prompt. The key/value maps keep retrieval over 16K-token prompts but do not help HELMET or LongBench, where a rank-80 LoRA does as well or better (Appendix L). More prefill loops do not help few-shot MATH-500 or exact recall in HELMET (34 items; Appendix N). Models have at most 3B parameters and four loops; benchmarks are English.

References

Kirill Afendulev, Alexey Dontsov, Elena Tutubalina, and Anton Korznikov. 2026. What Attention Recalls and Recurrence Controls in Hybrid Language Models. *arXiv preprint arXiv:2609.04434*.

Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium*

on Operating Systems Design and Implementation (OSDI), pages 117–134.

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Abdulrahman Alabdulkareem, Irene Tenison, Yang Zhang, Yada Zhu, Boris Katz, and Dan Gutfreund. 2026. Swarm Telepathy: Prefill-Free KV-Cache Transfer Across Families. *arXiv preprint arXiv:2610.15095*.

Sangmin Bae, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Seungyeon Kim, and Tal Schuster. 2025a. Relaxed Recursive Transformers: Effective Parameter Sharing with Layer-wise LoRA. In *International Conference on Learning Representations (ICLR)*. ArXiv:2410.20672.

Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyoun Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. 2025b. Mixture-of-Recursions: Learning Dynamic Recursive Depths for Adaptive Token-Level Computation. In *Advances in Neural Information Processing Systems (NeurIPS)*. ArXiv:2507.10524.

Andrea Banino, Jan Balaguer, and Charles Blundell. 2021. PonderNet: Learning to Ponder. *arXiv preprint arXiv:2107.05407*.

Payman Behnam, Debopam Sanyal, Pranav Ponnusamy, Kusum Meda Ravi, Pranav Sastry, Mani Chandana Bandaru, Alind Khare, Myungjin Lee, and Alexey Tumanov. 2026. BridgeKV: Large-Model Prefill and Small-Model Decoding for LLM Inference via KV Cache Translation. *arXiv preprint arXiv:2611.11761*.

Benjamin Bergner, Andrii Skliar, Amelie Royer, Tijmen Blankevoort, Yuki Asano, and Babak Ehteshami Bejnordi. 2024. Think Big, Generate Quick: LLM-to-SLM for Fast Autoregressive Decoding. *arXiv preprint arXiv:2402.16844*.

William Brandon, Mayank Mishra, Aniruddha Nrusimha, Rameswar Panda, and Jonathan Ragan-Kelley. 2024. Reducing Transformer Key-Value Cache Size with Cross-Layer Attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37.

Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, and Wen Xiao. 2024. PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling. *arXiv preprint arXiv:2406.02069*.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating Large Language Model Decoding with Speculative Sampling. *arXiv preprint arXiv:2302.01318*.

Luciano Del Corro, Allie Del Giorno, Sahaj Agarwal, Bin Yu, Ahmed Awadallah, and Subhabrata Mukherjee. 2023. SkipDecode: Autoregressive Skip Decoding with Batching and Caching for Efficient LLM Inference. *arXiv preprint arXiv:2307.02628*.

Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv preprint arXiv:2307.08691*.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. 2019. Universal Transformers. In *International Conference on Learning Representations (ICLR)*. ArXiv:1807.03819.

Xin Dong, Yonggan Fu, Shizhe Diao, Wonmin Byeon, Zijia Chen, Ameya Sunil Mahabaleshwarkar, Shih-Yang Liu, Matthijs Van Keirsbilck, Min-Hung Chen, Yoshi Suhara, Yingyan Lin, Jan Kautz, and Pavlo Molchanov. 2025. Hymba: A Hybrid-head Architecture for Small Language Models. In *International Conference on Learning Representations (ICLR)*. ArXiv:2411.13676.

Zhixu Du, Yuhan Cheng, Hai Li, and Yiran Chen. 2026. Not All Loops Cost the Same: Phase-Dependent Depth for Serving Looped Transformers. *arXiv preprint arXiv:2611.07211*.

Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. 2020. Depth-Adaptive Transformer. In *International Conference on Learning Representations (ICLR)*.

Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, Ahmed A. Aly, Beidi Chen, and Carole-Jean Wu. 2024. LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.

Siqi Fan, Xuezhi Fang, Xingrun Xing, Peng Han, Shuo Shang, and Yequan Wang. 2025a. Position-Aware Depth Decay Decoding (D^3): Boosting Large Language Model Inference Efficiency. In *Findings of the Association for Computational Linguistics: ACL 2025*. ArXiv:2503.08524.

Ying Fan, Yilun Du, Kannan Ramchandran, and Kangwook Lee. 2025b. Looped Transformers for Length Generalization. In *International Conference on Learning Representations (ICLR)*. ArXiv:2409.15647.

Maxim Fishman, Simeng Sun, Shantanu Acharya, Faisal Ladhak, Brian Chmiel, Santiago Akle Serano, Ron Banner, Boris Ginsburg, and Krishna C. Puvvada. 2026. Dory1.0-4B: A looped normalized-GPT. *arXiv preprint arXiv:2611.03998*.

Tianyu Fu, Zihan Min, Hanling Zhang, Jichao Yan, Guohao Dai, Wanli Ouyang, and Yu Wang. 2026. Cache-to-Cache: Direct Semantic Communication Between Large Language Models. In *International Conference on Learning Representations (ICLR)*. ArXiv:2510.03215.

Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. 2025. Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. In *Advances in Neural Information Processing Systems (NeurIPS)*. ArXiv:2502.05171.

Angeliki Giannou, Shashank Rajput, Jy yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. 2023. Looped Transformers as Programmable Computers. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *PMLR*, pages 11398–11442.

Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. 2024. Think Before You Speak: Training Language Models With Pause Tokens. In *International Conference on Learning Representations (ICLR)*.

Alex Graves. 2016. Adaptive Computation Time for Recurrent Neural Networks. *arXiv preprint arXiv:1603.08983*.

Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. 2025. Training Large Language Models to Reason in a Continuous Latent Space. In *Conference on Language Modeling (COLM)*. ArXiv:2412.06769.

Junhui He, Zhihui Fu, Jun Wang, and Qingan Li. 2026. POP: Prefill-Only Pruning for Efficient Large Model Inference. In *Findings of the Association for Computational Linguistics: ACL 2026*. ArXiv:2602.03295.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations (ICLR)*. ArXiv:2106.09685.

Zhiheng Hu, Yixun Wei, Jian Zhou, Yizhuang Zhou, Ji Li, Xing Chen, Yang Li, Bojun Wang, Yibo Zhu, Xiangyu Zhang, and Daxin Jiang. 2026. KITE: KV-Invariant Transformer Expansion for Efficient Agentic LLM Scaling. *arXiv preprint arXiv:2609.27294*.

Benhao Huang, Chufan Shi, Junlin Chen, Shicheng Wen, Zhengzhong Liu, Eric Xing, and Xuezhe Ma. 2026. Towards Looped Models Done Right, Part II: Rethinking at Fixed Points. *arXiv preprint arXiv:2610.06833*.

Jungo Kasai, Nikolaos Pappas, Hao Peng, James Cross, and Noah A. Smith. 2021. Deep Encoder, Shallow Decoder: Reevaluating Non-autoregressive Machine Translation. In *International Conference on Learning Representations (ICLR)*. ArXiv:2006.10369.

Woojeong Kim, Junxiong Wang, Jing Nathan Yan, Mohamed Abdelfattah, and Alexander M. Rush. 2025. OverFill: Two-Stage Models for Efficient Language Model Decoding. *arXiv preprint arXiv:2508.08446*.

Andreas Kirsch. 2026. LSTM-Style Looped Transformers with Shared KV Caches. *arXiv preprint arXiv:2610.33360*.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*.

Faisal Ladhak, Shantanu Acharya, Santiago Akle Serano, Simeng Sun, Boris Ginsburg, and Krishna C Puvvada. 2026. One Model to Loop Them All: Elastic Latent Recurrence for Language Models. *arXiv preprint arXiv:2610.12582*.

Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. 2020. ALBERT: A Lite BERT for Self-supervised Learning of Language Representations. In *International Conference on Learning Representations (ICLR)*.

John Langford, Nathan Godey, Giovanni Monea, Yoav Artzi, Harry Dong, Ying Fan, Gustavo de Rosa, and Zheng Zhan. 2026. Almost Free State Prediction Separation. *arXiv preprint arXiv:2609.03807*.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast Inference from Transformers via Speculative Decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*.

Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. SnapKV: LLM Knows What You are Looking for Before Generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37.

Liming Liu, Mingze Wang, and Tuo Zhao. 2026. Decode-Branch Transformers: Decoupling the Primary Prefill Path from Additional Decode Computation. *arXiv preprint arXiv:2608.12385*.

Mingyuan Ma, Yangmin Li, Harris Nover, Zheyu Fu, Po-Han Huang, Alex Settle, and Ashwin Nanjappa. 2026. Shallow encoding emits deep memories. *arXiv preprint arXiv:2611.06084*.

Giovanni Monea, Keshav Ramji, Yousef El-Kurdi, Luis A. Lastras, Yoav Artzi, Nathan Godey, and Ramón Fernandez Astudillo. 2026. The Surprising Effectiveness of Shared Memory in Looped Transformers. *arXiv preprint arXiv:2610.02383*.

James O' Neill and Fergal Reid. 2026. Looped Latent Attention: Cross-Loop KV Compression for Looped Transformers. *arXiv preprint arXiv:2607.15456*.

Jungsuk Oh, Hyeseo Jeon, Hyunjune Ji, Kyongmin Kong, and Jay-Yoon Lee. 2026. Shallow Prefill, Deep Decoding: Efficient Long-Context Inference via Layer-Asymmetric KV Visibility. *arXiv preprint arXiv:2605.06105*.

Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *51st ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, pages 118–132.

Hayden Prairie, Jacob M. Ryan, Taylor Berg-Kirkpatrick, and Daniel Y. Fu. 2026. BeNice! Closing Looping's Inference-Memory Gap via KV-Sharing. *arXiv preprint arXiv:2610.31633*.

Aurick Qiao, Zhewei Yao, Samyam Rajbhandari, and Yuxiong He. 2025. SwiftKV: Fast Prefill-Optimized Inference with Knowledge-Preserving Model Transformation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. ArXiv:2410.03960.

Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST)*, pages 155–170. ArXiv:2407.00079.

Haomiao Qiu, Yining Tang, Zijia Song, Ruizi Han, Ziyue Qiao, and Miao Zhang. 2026. CacheLink: Functional KV Transport for Zero-Prefill Cross-Model Context Handoff. *arXiv preprint arXiv:2610.15509*.

David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. 2024. Mixture-of-Depths: Dynamically Allocating Compute in Transformer-Based Language Models. *arXiv preprint arXiv:2404.02258*.

Aishwarya P S, Pranav Ajit Nair, Yashas Samaga, Toby Boyd, Sanjiv Kumar, Prateek Jain, and Praneeth Netrapalli. 2024. Tandem Transformers for Inference Efficient LLMs. *arXiv preprint arXiv:2402.08644*.

Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. 2025. Reasoning with Latent Thoughts: On the Power of Looped Transformers. In *International Conference on Learning Representations (ICLR)*. ArXiv:2502.17416.

Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. 2022. Confident Adaptive Language Modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Kristian Schwethelm, Daniel Rueckert, and Georgios Kaissis. 2026. Depth-adaptive Inference of Looped Language Models via Continuous Depth Batching. *arXiv preprint arXiv:2608.09444*.

Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. *arXiv preprint arXiv:1911.02150*.

Zhenyi Shen, Junru Lu, Wei Liu, Jiangnan Ye, Jiazheng Li, Di Yin, Xing Sun, Tameem Adel, Yali Du, Lin Gui, and Yulan He. 2026. On the opportunities and challenges of prefill-to-autoregressive cross-model transfer. https://github.com/zhenyi4/platour. Paper and code.

Dingjie Song, Zhiling Yan, Yuxuan Zhang, and Lichao Sun. 2026. LEV: Thinking Fast and Deep with Looped Decision Models. *arXiv preprint arXiv:2610.20951*.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. 2024. RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing*, 568:127063. ArXiv:2104.09864.

Yutao Sun, Li Dong, Tianzhu Ye, Shaohan Huang, Jianyong Wang, and Furu Wei. 2026. Universal YOCO for Efficient Depth Scaling. *arXiv preprint arXiv:2604.01220*.

Yutao Sun, Li Dong, Yi Zhu, Shaohan Huang, Wenhui Wang, Shuming Ma, Quanlu Zhang, Jianyong Wang, and Furu Wei. 2024. You Only Cache Once: Decoder-Decoder Architectures for Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*. ArXiv:2405.05254.

Victor Conchello Vendrell, Arnau Padres Masdemont, Niccolò Grillo, Jordi Ros-Giralt, Arash Behboodi, and Fabio Valerio Massoli. 2026. Memory-Efficient Looped Transformer: Decoupling Compute from Memory in Looped Language Models. *arXiv preprint arXiv:2605.07721*.

Simon P. Villani. 2026. LatentPort: Beyond KV Cache - Cross-Model Transfer of Recurrent Memory in Hybrid Language Models: A 4B-to-9B Hybrid-State Handoff Without Target Prefix Replay. *arXiv preprint arXiv:2609.25053*.

Bohong Wu, Mengzhao Chen, Xiang Luo, Shen Yan, Qifan Yu, Fan Xia, Tianqi Zhang, Hongrui Zhan, Zheng Zhong, Xun Zhou, Siyuan Qiao, and Xingyan Bin. 2025a. Parallel Loop Transformer for Efficient Test-Time Computation Scaling. *arXiv preprint arXiv:2510.24824*.

Bohong Wu, Shen Yan, Sijun Zhang, Jianqiao Lu, Yutao Zeng, Ya Wang, and Xun Zhou. 2025b. Efficient Pretraining Length Scaling. *arXiv preprint arXiv:2504.14992*.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient Streaming Language Models with Attention Sinks. In *International Conference on Learning Representations (ICLR)*.

Ji Xin, Raphael Tang, Jaejun Lee, Yaoliang Yu, and Jimmy Lin. 2020. DeeBERT: Dynamic Early Exiting for Accelerating BERT Inference. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*.

Jian Yang, Shawn Guo, Wei Zhang, Tianyu Zheng, Yaxin Du, Haau-Sing Li, Jiajun Wu, Yue Song, Yan Xing, Qingsong Cai, Zelong Huang, Chuan Hao, Ran Tao, Xianglong Liu, Wayne Xin Zhao, Mingjie Tang, Weifeng Lv, Ming Zhou, and Bryan Dai. 2026. LoopCoder-v2: Only Loop Once for Efficient Test-Time Computation Scaling. *arXiv preprint arXiv:2606.18023*.

Songlin Yang, Jan Kautz, and Ali Hatamizadeh. 2025. Gated Delta Networks: Improving Mamba2 with Delta Rule. In *International Conference on Learning Representations (ICLR)*. ArXiv:2412.06464.

Wanqi Yang and Shiwei Liu. 2026. FlashLoop: Fast and memory-efficient looped transformers via lazy updates. *arXiv preprint arXiv:2609.29812*.

Howard Yen, Tianyu Gao, Minmin Hou, Ke Ding, Daniel Fleischer, Peter Izsak, Moshe Wasserblat, and Danqi Chen. 2025. HELMET: How to Evaluate Long-Context Language Models Effectively and Thoroughly. In *International Conference on Learning Representations (ICLR)*. ArXiv:2410.02694.

Vincent-Daniel Yun, Woosang Lim, Haneul Yoo, Sungjoo Yoo, Murali Annavaram, and Sai Praneeth Karimireddy. 2026. Prefill-Free Cross-Family KV Cache Transfer for Heterogeneous Multi-Agent LLMs. *arXiv preprint arXiv:2609.32259*.

Jiamu Zhang and Hanjie Chen. 2026. BellowsKV: One Latent for Many Layers, Expanded on Demand. *arXiv preprint arXiv:2610.35023*.

Yudong Zhang, Wei Ding, Kaili Zheng, Yuchen Ding, and Zhenyu Zhang. 2026. Accordion: Elastic-Depth Inference via Fixed-Prefix Layer Looping. *arXiv preprint arXiv:2610.32492*.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H_2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36.

Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, Lu Li, Jiajun Shi, Kaijing Ma, Shanda Li, Taylor Kergan, Andrew Smith, Xingwei Qu, Mude Hui, Bohong Wu, and 14 others. 2025. Scaling Latent Reasoning via Looped Language Models. *arXiv preprint arXiv:2510.25741*.

Zayd Muhammad Kawakibi Zuhri, Muhammad Farid Adilazuarda, Ayu Purwarianti, and Alham Fikri Aji. 2025. MLKV: Multi-Layer Key-Value Heads for Memory Efficient Transformer Decoding. In *Findings of the Association for Computational Linguistics: NAACL 2025*.

Guide to the appendix. Appendices A–C give the full data behind Figures 2–5 and Section 5.5; Appendix D adds results on the reasoning models and decision models; Appendices E–H give method, training and evaluation details; the remaining appendices give further checks, and Appendix O lists what is released. In the appendix, "LoRA-distilled" is Distilled and "loop adapter" or "aligned adapter" is LoopBridge; "one cache" is its lighter variant in which every decode loop reads the last prefill loop. In every figure ours is orange and the original model grey. Table 3 lists which implementation and which measurement each result uses.

result	LoopBridge form	measurement
Tables 1, 2; Figures 4, 5	per-loop caches, maps as in training	accuracy, full evaluation sets
Figures 6, 7	per-loop caches, maps folded in	batch 1, replayed requests
Figures 8, 14	per-loop caches, maps folded in	continuous batching, wall time
Figures 11, 12, 15	one shared cache	accuracy; decode microbenchmark

Table 3: **Implementation behind each result.** The per-loop forms compute the same function in exact arithmetic (Section 3); the one-cache LoopBridge is a separately trained variant.

A Per-Request Time

Table 4 gives the per-request distributions behind Figure 2: medians and 90th percentiles of the output length, the time per request and the prefill

share, over the same 100 requests per task for every configuration (batch 1, one B300, greedy). At the 90th percentile the prefill share stays below 5% on GSM8K, MATH-500 and BBH; on MMLU-Pro, where many answers are short, it reaches 5–17%. The time per request follows the output length, whose 90th percentile is up to about four times its median within a task.

Every time is the mean of two passes over the same requests in opposite orders on a machine running nothing else; the answers are identical in both. Between the passes the mean time per request of a configuration and task differs by 5.9% at the median and by up to 22.5% (128 configuration-task pairs). An earlier pass, on a machine that also ran evaluations, is not used. Folding the maps changes floating-point rounding, so greedy answers can differ: on the four tasks Ours 4/1 scores 41.2 with folded maps and 43.0 with per-step maps.

B Where the Gain Stops

For each benchmark and decode depth, the stopping depth m^* is the first sampled prefill depth that none of the next three sampled depths beats by more than 0.5 points on a development half of the items (split by a hash of the item id); the accuracy at m^* and at the largest evaluated depth is reported on the other half (Table 6). The rule has no interval or equivalence test. Settings where no evaluated depth meets it, because accuracy still rises or too few larger depths were evaluated, are left out: for the main adapter MMLU, MMLU-Pro, BBH and ARC-C at $n = 1$, MMLU-Pro and ARC-C at $n = 2$, and HellaSwag and Winogrande at both; for the adapter trained up to 16, MMLU-Pro and ARC-C at $n = 2$. Over three training seeds of that adapter, 16 settings have a stop in every seed; the three stops are equal in 8 of them, within one sampled depth in 11 and all at two to four loops in 9, and MATH-500 stops at one prefill loop in every seed.

Training the prefill as well. In the adapters above the prefill loops stay frozen, so the stop could come from prefill states that were never trained for more loops. We therefore train two adapters that differ only in this: both on $m \in \{1, 2, 3, 4, 6, 8\}$ and $n \leq 4$ with training prompts cut to their last 2,048 tokens (to bound the memory of back-propagating through eight prefill loops; evaluation is not cut), one with the prefill frozen and one with an additional rank-16 LoRA on the attention of the prefill loops (Table 5: seed 1). Training the prefill does not move the stop: of the 16 settings with a stop under both, it comes later with the trained prefill in 2, at the same depth in 8 and earlier in 6, and both curves peak at three or four loops on most benchmarks. Two further training seeds of both adapters agree: of the 13 settings with a stop in all six runs, the trained prefill stops later than the frozen one in 2, 4 and 4 settings in the three seeds, at the same depth in 6, 6 and 5, and later in all three seeds in none. The trained prefill mainly helps long prompts: on RULER-16K the best accuracy rises from 63.7 to 66.0 at one decode loop and from 75.2 to 81.0 at two, and beyond eight loops, outside both adapters' training range, it falls more slowly; 4-shot MATH-500 instead collapses at twelve loops with the trained prefill.

C Prefill Depth on Ouro-1.4B

Table 7 repeats the sweep of Section 5.4 on Ouro-1.4B with its loop adapter (trained on $m, n \leq 4$), on the same benchmarks and protocol.

D Additional Results

D.1 More prefill loops, fewer decode loops

Figure 11 gives the reasoning model per benchmark: Ouro-2.6B-Thinking at prefill 4 / decode 2. With the same number of decode loops as Original 2/2, Ours 4/2 scores higher on all eight benchmarks, by up to 13 points on HELMET and 7 on MMLU-Pro and BBH; against Original 4/4, which decodes with twice the loops, it is within two points on six of the eight. The prefill loops themselves carry the gain: the same adapter at Ours 2/2 is up to 8 points lower on MMLU-Pro and HELMET (Appendix J). Ours 4/2 also scores above Original 2/2 distilled on the same data on all eight (Table 13).

D.2 End-to-end time

We time whole answers at batch 1, prefill included (Table 14 in Appendix K). On Ouro-2.6B-Thinking, prefill 4 / decode 2 answers 192 timed GSM8K problems in $0.59\times$ [0.52, 0.66] the time of Original 4/4. On all 1,319 problems it scores 93.3 against 94.2, a paired difference of -1.0 [−2.0, 0.1]. Prefilling with 4 rather than 2 loops before decoding with two changes the answer time by $1.02\times$ [0.92, 1.13], and prefill 4 / decode 2 takes $1.20\times$ [1.05, 1.38] the time of the distilled Original

task	configuration	input	output	output p90	s / request	p90	prefill %	p90
GSM8K	Ours 1/1	318	88	142	4.1	6.6	2.38	3.55
GSM8K	Ours 1/2	318	96	143	9.3	13.7	1.64	2.32
GSM8K	Original 1/1	318	90	171	3.2	6.0	2.65	4.50
GSM8K	Original 2/2	318	94	157	6.5	10.8	2.23	3.20
GSM8K	Original 4/4	318	94	145	13.1	20.3	2.09	3.14
MATH-500	Ours 1/1	743	601	1024	27.1	45.9	0.40	0.81
MATH-500	Ours 1/2	743	491	1024	48.9	101.8	0.32	0.61
MATH-500	Original 1/1	743	631	1024	25.9	42.0	0.36	0.81
MATH-500	Original 2/2	743	324	1024	22.0	70.1	0.64	1.56
MATH-500	Original 4/4	743	260	1024	36.3	139.9	0.78	1.83
BBH	Ours 1/1	984	168	332	7.9	16.1	1.46	2.84
BBH	Ours 1/2	984	164	333	14.7	30.4	1.07	2.03
BBH	Original 1/1	984	168	1024	5.9	35.6	1.56	3.77
BBH	Original 2/2	984	168	344	12.7	26.0	1.32	2.77
BBH	Original 4/4	984	156	343	21.6	47.2	1.28	2.33
MMLU-Pro	Ours 1/1	1331	57	170	2.7	8.0	4.52	7.43
MMLU-Pro	Ours 1/2	1331	52	214	4.9	20.1	3.23	5.45
MMLU-Pro	Original 1/1	1331	44	143	1.6	5.0	6.70	17.15
MMLU-Pro	Original 2/2	1331	60	157	4.2	10.9	3.84	6.56
MMLU-Pro	Original 4/4	1331	60	248	8.3	34.6	3.55	5.80

Table 4: **Per-request time, Ouro-2.6B.** Medians unless marked p90 (90th percentile); input and output in tokens; prefill %: prefill time over total time per request.

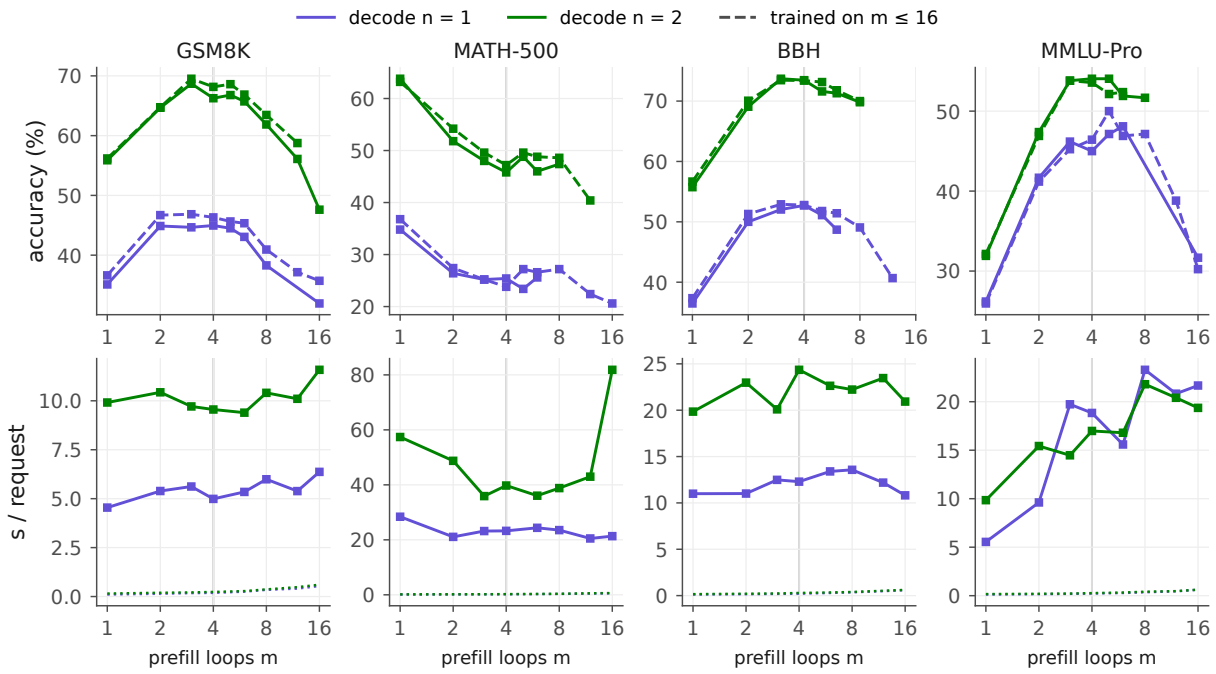


Figure 10: **Figure 5 in full.** Ouro-2.6B, same weights at every m; decode $n = 1$ (violet) and $n = 2$ (green). Top: accuracy on the evaluation sets of Appendix H, solid for the adapter trained on $m \leq 4$ (it extrapolates right of the grey line at $m = 4$), dashed for the one trained on $m \in \{1, 2, 3, 4, 6, 8, 12, 16\}$. Bottom: time per request on 100 timed requests per task, maps applied at each decode step (dotted: its prefill part).

2/2, at similar accuracy on these problems: the saving comes from decoding with two loops, and what the extra prefill loops add is the accuracy of Section D.1. On MATH-500 the saving is smaller: $0.63\times$ the time of Original 4/4 but $1.22\times$ that of Original 2/2 (Appendix K). A smaller model is not the same trade: original Ouro-1.4B-Thinking 4/4 takes about as long per answer (65.2 against 69.6 seconds), and Ouro-2.6B-Thinking at prefill 4 / decode 2 scores higher on five of the eight benchmarks of Figure 11, by 3.5 to 3.8 points on MMLU-Pro, ARC-Challenge and BBH.

benchmark	n	prefill	m=1	2	3	4	5	6	8	12	16	m^*
MMLU	1	frozen	51.5	63.3	66.4	**67.0**	66.3	65.0	62.4	–	–	3
MMLU	1	trained	52.5	64.5	67.5	**68.1**	67.3	66.5	64.8	–	–	4
MMLU-Pro	1	frozen	27.1	42.9	**48.1**	46.9	47.4	47.4	47.6	40.7	33.1	–
MMLU-Pro	1	trained	28.6	39.3	**45.5**	44.8	43.6	42.9	44.3	40.7	34.8	–
BBH 3-shot	1	frozen	37.0	50.9	52.3	**53.3**	52.6	52.2	49.9	39.2	–	4
BBH 3-shot	1	trained	37.5	49.4	53.0	**54.6**	53.1	52.2	52.5	47.6	–	4
ARC-C	1	frozen	52.2	59.3	61.8	**63.1**	62.3	62.5	61.2	–	–	4
ARC-C	1	trained	53.0	59.3	**62.0**	61.8	61.3	61.7	61.4	–	–	3
GSM8K	1	frozen	35.3	45.4	45.0	**45.5**	44.0	43.8	40.0	36.2	32.9	3
GSM8K	1	trained	35.8	**45.6**	45.6	45.2	43.5	43.9	40.0	38.5	35.6	2
MATH-500 4-shot	1	frozen	**36.0**	25.0	25.4	27.8	24.6	26.0	26.8	21.0	19.4	1
MATH-500 4-shot	1	trained	**35.4**	24.4	24.6	24.2	25.8	25.8	25.0	5.2	7.8	1
HumanEval+	1	frozen	52.4	62.2	**65.8**	64.6	63.4	62.8	57.3	56.7	45.1	3
HumanEval+	1	trained	49.4	58.5	**65.8**	65.2	64.6	63.4	60.4	56.7	53.7	4
MBPP+	1	frozen	49.5	55.3	55.8	56.9	**58.2**	57.4	55.8	54.2	50.0	5
MBPP+	1	trained	48.1	**57.4**	54.5	56.6	56.1	56.1	54.8	55.3	51.9	2
RULER-16K	1	frozen	52.3	59.0	61.1	**63.7**	60.5	59.9	58.8	53.1	41.2	4
RULER-16K	1	trained	52.4	62.6	**66.0**	65.8	63.9	64.2	63.8	59.5	52.3	3
MMLU	2	frozen	53.4	67.5	72.9	**73.2**	72.5	71.2	68.4	–	–	4
MMLU	2	trained	53.7	68.5	73.0	**73.6**	72.9	72.5	70.4	–	–	4
MMLU-Pro	2	frozen	30.0	45.5	**54.5**	53.8	**54.5**	53.8	51.9	44.0	38.3	5
MMLU-Pro	2	trained	31.4	44.3	52.4	53.8	51.9	**54.3**	50.5	48.1	42.4	4
BBH 3-shot	2	frozen	55.4	69.8	73.3	**73.6**	72.5	70.6	69.7	62.0	–	–
BBH 3-shot	2	trained	54.6	69.4	73.1	73.6	73.5	**73.7**	72.5	66.8	–	–
ARC-C	2	frozen	58.5	62.5	64.4	**64.8**	64.8	64.6	64.4	–	–	3
ARC-C	2	trained	58.2	62.2	64.8	65.0	65.1	64.5	**65.2**	–	–	3
GSM8K	2	frozen	55.9	65.5	**69.1**	66.1	66.6	64.8	61.9	56.0	47.3	3
GSM8K	2	trained	56.5	63.8	**68.0**	67.6	66.9	65.6	62.2	57.9	47.1	3
MATH-500 4-shot	2	frozen	**63.4**	56.4	49.4	48.6	49.0	48.0	50.6	42.4	–	1
MATH-500 4-shot	2	trained	**63.4**	52.8	49.6	48.2	49.0	49.4	49.2	12.4	–	1
HumanEval+	2	frozen	65.2	67.7	**71.3**	68.9	68.3	65.2	65.2	62.8	54.3	3
HumanEval+	2	trained	64.6	65.8	**68.9**	65.8	67.7	65.2	66.5	65.8	59.8	3
MBPP+	2	frozen	56.4	60.9	**63.8**	63.5	63.5	62.4	59.5	58.5	53.2	5
MBPP+	2	trained	58.2	61.1	62.4	**64.5**	63.5	64.3	64.0	61.6	56.6	4
RULER-16K	2	frozen	56.1	67.5	**75.2**	73.4	70.2	69.9	69.2	61.7	52.3	3
RULER-16K	2	trained	57.8	71.7	80.3	**81.0**	75.7	74.7	74.6	70.5	60.7	3

Table 5: **Training the prefill loops as well.** Accuracy (%) against prefill loops m for two adapters trained identically on $m \leq 8$ with prompts cut to 2,048 tokens, one with the prefill frozen and one with a LoRA on the prefill attention; items both share; best m of each row in bold; m^*: stopping depth (–: none); "–" in an m column: not evaluated.

D.3 Looped decision models: both directions

Untying is not specific to generation. Lev (Song et al., 2026) turns looped language models into decision models: they read a long context, then score the options of a question with a learned head after every loop, without generating text. We prefill the context with m loops, let each of the n decision loops attend to the last prefill loop's KV, and continue Lev's own LoRA and scoring head, once for every m; the control continues the same weights on the same data for the same number of updates with Lev's Original n/n forward (Appendix M).

In these runs the sign of $m - n$ gives the direction of the difference (Figure 9, Table 20). With more prefill than decision loops the model is significantly above the control on 12 of 13 (m, n) pairs, most with one decision loop, where prefilling with all of the backbone's loops gains $+5.3$ [3.4, 7.2] on Lev-1.4B, $+7.5$ [5.7, 9.5] on Lev-2.6B and $+4.4$ [2.8, 6.1] on Lev-3B. With fewer prefill than decision loops it is below the control on all 13 pairs (significantly on 11), by up to 13.0 points (-13.0 [-14.7, -11.3]; Lev-2.6B, prefill 1, four decision loops). The fewer-prefill direction replicates over four training seeds: every seed is below the control on all 13 pairs, and on 10 of them significantly in every seed; the more-prefill comparison at the backbone's full loop count is replicated over four seeds and against a control trained twice as long (Appendix M). As in the task map, the context gains from many loops and the decision can then stop early.

E Method Details

adapter	benchmark	n	m^*	test m=1	test m^*	test largest
trained ≤ 4	GSM8K	1	2	36.5	46.0	32.3 (16)
trained ≤ 4	MATH-500 4-shot	1	1	36.2	36.2	28.4 (6)
trained ≤ 4	HumanEval+	1	3	43.2	63.0	40.7 (16)
trained ≤ 4	MBPP+	1	4	48.0	54.0	49.5 (16)
trained ≤ 4	RULER-16K	1	4	52.2	65.6	40.6 (16)
trained ≤ 4	MMLU	2	4	52.7	73.2	68.0 (8)
trained ≤ 4	BBH 3-shot	2	3	55.4	72.8	69.6 (8)
trained ≤ 4	GSM8K	2	3	56.4	69.2	47.9 (16)
trained ≤ 4	MATH-500 4-shot	2	1	66.2	66.2	46.7 (8)
trained ≤ 4	HumanEval+	2	3	59.3	69.1	50.6 (16)
trained ≤ 4	MBPP+	2	5	55.6	62.1	50.5 (16)
trained ≤ 4	RULER-16K	2	3	57.3	78.2	52.3 (16)
trained ≤ 16	MMLU	1	3	50.7	66.5	61.3 (8)
trained ≤ 16	MMLU-Pro	1	5	28.0	48.9	31.6 (16)
trained ≤ 16	BBH 3-shot	1	3	36.9	51.8	39.9 (12)
trained ≤ 16	ARC-C	1	3	50.1	62.7	62.5 (8)
trained ≤ 16	GSM8K	1	2	39.0	47.0	36.9 (16)
trained ≤ 16	MATH-500 4-shot	1	1	36.6	36.6	19.1 (16)
trained ≤ 16	HumanEval+	1	5	48.1	59.3	48.1 (16)
trained ≤ 16	MBPP+	1	2	48.0	52.0	50.0 (16)
trained ≤ 16	RULER-16K	1	2	52.8	60.7	44.5 (16)
trained ≤ 16	MMLU	2	3	52.9	72.5	70.8 (6)
trained ≤ 16	BBH 3-shot	2	3	58.1	72.7	69.7 (8)
trained ≤ 16	GSM8K	2	3	57.3	70.5	59.2 (12)
trained ≤ 16	MATH-500 4-shot	2	1	65.8	65.8	41.6 (12)
trained ≤ 16	HumanEval+	2	3	61.7	65.4	53.1 (12)
trained ≤ 16	MBPP+	2	4	57.1	60.1	55.0 (12)
trained ≤ 16	RULER-16K	2	3	58.9	77.0	53.5 (16)

Table 6: **Stopping depth of the prefill gain.** "trained ≤ 4": the main adapter (beyond four it extrapolates); "trained ≤ 16": the adapter trained on $m \in \{1, 2, 3, 4, 6, 8, 12, 16\}$. Largest: the largest evaluated depth, in parentheses.

benchmark	n	m=1	2	3	4	5	6	8	12	16
MMLU	1	42.8	56.9	**63.7**	63.6	60.2	59.0	56.1	46.1	31.9
MMLU-Pro	1	20.0	30.0	36.4	**36.7**	29.1	30.0	28.1	20.2	12.1
BBH 3-shot	1	**33.1**	9.5	9.3	13.4	9.3	8.6	14.1	18.5	7.5
ARC-C	1	45.4	52.2	54.9	**56.3**	52.7	53.8	52.7	45.6	35.6
GSM8K	1	21.3	27.2	**29.6**	25.7	15.2	17.6	21.2	18.3	9.9
MATH-500 4-shot	1	**28.0**	24.0	27.6	24.2	15.2	21.8	15.2	10.6	9.8
HumanEval+	1	40.2	39.6	**46.3**	39.6	29.9	37.2	34.1	28.1	0.0
MBPP+	1	42.3	47.4	**49.2**	47.6	38.9	43.9	43.9	33.1	0.0
RULER-16K	1	43.4	45.9	45.1	**47.2**	36.8	43.1	43.5	40.0	29.2
MMLU	2	43.6	60.6	66.2	**66.7**	65.2	62.6	60.1	50.2	35.6
MMLU-Pro	2	24.3	36.9	44.3	43.8	44.3	**45.5**	38.8	34.3	23.6
BBH 3-shot	2	46.2	52.7	**53.1**	49.4	47.5	32.6	39.8	36.2	16.6
ARC-C	2	51.7	56.2	58.2	**58.7**	58.1	57.7	56.3	50.3	39.6
GSM8K	2	41.1	46.6	**53.3**	51.1	40.8	39.7	43.0	39.5	23.6
MATH-500 4-shot	2	53.8	**57.4**	50.8	53.2	43.8	46.0	36.6	23.2	22.0
HumanEval+	2	61.0	56.1	**63.4**	62.8	54.3	51.2	50.0	47.0	0.0
MBPP+	2	53.2	55.8	**57.7**	57.1	49.2	50.3	49.5	42.6	0.0
RULER-16K	2	50.6	**59.6**	49.9	53.5	45.7	47.8	50.5	42.1	35.3

Table 7: **Prefill depth on Ouro-1.4B.** Accuracy (%) against prefill loops m at decode n; best m of each row in bold; –: not evaluated.

We describe LoopBridge for Ouro (Figure 3); the fixed-depth adapter is the same code with the maps fixed at the identity and m fixed.

Prefill and decode. "Ours m/n" prefills the prompt with m loops and decodes every answer token with n loops; Original n/n is the unmodified model. The frozen model prefills the prompt with m loops and keeps the prompt keys and values its decode loops read: with aligned caches (Ours), decode loop k reads prefill loop $\max(m - n + k, 1)$; in the one-cache variant every decode loop reads the last prefill loop. The decoder is the same frozen model with rank-16 LoRA (Hu et al., 2022) on its attention projections, running n loops per

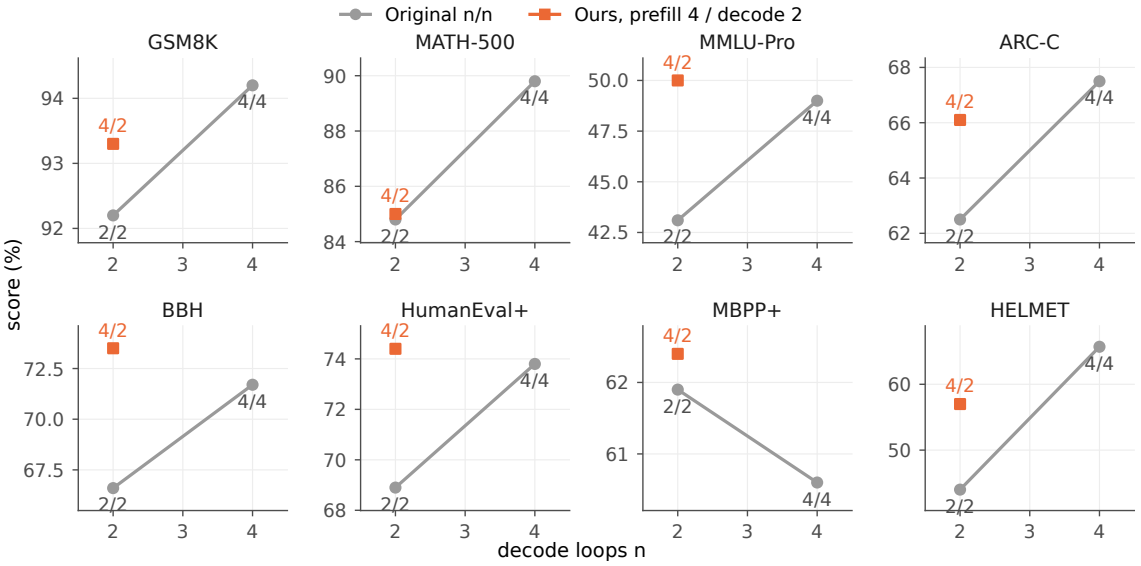


Figure 11: **On a reasoning model, two decode loops with four prefill loops beat two loops everywhere.** Ouro-2.6B-Thinking, score against the number of decode loops. Grey: Original 2/2 and 4/4. Orange: LoopBridge (one cache) with prefill 4 / decode 2. Same items for all three in each panel; HELMET is averaged over its categories. Table 12 gives every score, the paired intervals and the same adapter at prefill 2 / decode 2.

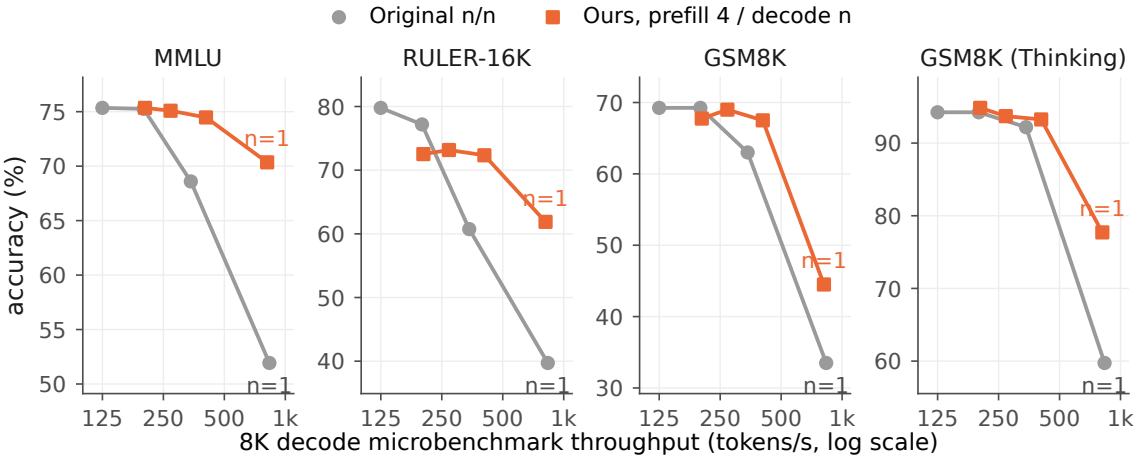


Figure 12: **At the same decode throughput, untying is far more accurate.** Original n/n (grey) and LoopBridge (one cache) with prefill 4 / decode n (orange); each line runs from $n = 1$ (labelled, right) to $n = 4$ (left; Thinking adapter $n = 1$–3). Throughput is a separate decode microbenchmark: best decode tokens/s over batch sizes at 8K context on one B300 (same for the base and Thinking models, which share the architecture); one stored prompt KV cache admits larger batches as n grows. Ouro-2.6B on MMLU (20 questions per subject), RULER-16K (260 items) and GSM8K (400 problems); Ouro-2.6B-Thinking on all 1,319 GSM8K problems.

token.

RoPE-commuting maps. Decode loop k reads the prompt KV cache through a map per (loop, layer, KV head). The key map is one complex scale c_j per rotary frequency pair j (Su et al., 2024): viewing the pair of key coordinates (k_{2j}, k_{2j+1}) as a 2-D vector, $c_j = r\,e^{i\phi}$ stretches it by r and turns it by ϕ. Writing a key pair rotated for position s as a complex number $\tilde{k}_j = e^{is\theta_j} k_j$, scaling commutes with the rotation, $c_j\tilde{k}_j = e^{is\theta_j}(c_j k_j)$, and $\mathrm{Re}\langle q_j, c_j\tilde{k}_j\rangle = \mathrm{Re}\langle \bar{c}_j q_j, \tilde{k}_j\rangle$: the map is applied to the query and, in this per-step form, the stored cache is never rewritten (the timed form of Section 3 instead folds both maps into the cache once at prefill). The value map is a full linear map W_k per head; because attention is linear in the values, $\sum_i a_i W_k v_i = W_k \sum_i a_i v_i$ is applied once to the attention output over the prompt. Prompt and generated tokens share one softmax: the two

	MMLU	MMLU-P	BBH	ARC-C	HSwag	Wino	GSM8K	MATH	HE+	MBPP+	R-16K
native 1/1	51.5	26.9	30.9	47.7	69.0	61.1	32.4	32.2	44.5	49.2	41.4
1 / 1	51.2	26.2	36.5	52.5	69.0	65.3	35.1	**34.8**	46.3	49.2	52.7
2 / 1	63.2	41.7	50.0	60.0	72.3	67.1	44.9	26.4	59.1	54.8	60.2
3 / 1	66.4	**46.2**	52.0	61.8	**73.1**	**68.2**	44.7	25.2	**65.9**	**56.3**	62.4
4 / 1	**67.1**	45.0	**52.7**	**62.8**	72.7	68.0	**45.0**	25.4	65.2	**56.3**	**64.7**
native 2/2	67.7	45.0	66.6	62.2	77.8	71.9	62.9	50.6	66.5	61.1	60.8
1 / 2	53.4	31.9	55.7	58.5	74.6	68.0	55.9	**63.8**	61.6	56.3	57.1
2 / 2	67.6	47.4	69.1	62.4	77.5	73.1	64.7	51.8	66.5	61.6	68.6
3 / 2	72.9	53.8	**73.7**	64.3	78.1	**75.8**	**68.7**	48.0	**73.2**	63.2	**76.1**
4 / 2	**73.5**	**54.0**	73.5	**65.0**	**78.3**	75.1	66.3	45.8	70.1	**64.3**	73.5
native 3/3	73.5	55.5	77.7	65.1	79.2	77.1	69.1	54.4	72.0	64.3	78.7
1 / 3	53.0	32.6	63.1	58.3	76.0	68.4	60.0	**68.8**	67.1	58.2	58.8
2 / 3	67.4	46.2	74.0	63.5	78.4	74.3	66.2	62.4	68.3	60.6	68.6
3 / 3	73.6	53.8	**77.9**	65.0	79.1	**76.7**	**69.3**	55.8	**72.6**	63.5	79.2
4 / 3	**74.3**	**56.7**	77.7	**66.2**	**79.2**	76.3	68.4	53.0	69.5	**63.8**	**79.8**
native 4/4	74.6	55.2	80.4	66.5	79.4	77.7	69.4	53.6	69.5	64.5	81.3
1 / 4	53.0	32.9	64.8	58.0	76.4	69.1	61.2	**69.6**	67.7	59.8	57.8
2 / 4	67.7	45.5	74.3	62.6	78.4	74.2	67.0	64.8	67.7	62.2	66.7
3 / 4	73.6	**53.8**	77.8	65.4	79.1	76.5	**68.8**	59.6	**71.3**	**63.8**	79.1
4 / 4	**74.5**	**53.8**	**79.1**	**66.1**	**79.4**	**77.0**	**68.8**	54.6	68.9	63.2	**81.0**

Table 8: **Every prefill / decode cell of LoopBridge.** Ouro-2.6B, accuracy (%) of Original n/n and of LoopBridge at prefill m / decode n on the benchmarks of Table 1, each column on the items all available configurations share (Figure 4); bold: the best m at each n.

```
# blocks: frozen looped block, L layers; m/n: prefill/decode loops
# c[k][l] = 1+a+1j*b per RoPE pair; W[k][l] = I+delta (init: I)
# LoRA on q, k, v, o is active only while decoding.

def prefill(prompt, m): # prompt minus its last token
    h = embed(prompt)
    for j in range(m):
        for l in range(L):
            h, K[j][l], V[j][l] = blocks[l](h) # every loop's K, V
    return K, V

def attend(q, K, V, kv_answer, c, W): # layer l, decode loop k
    qc = complex_mul(q, c.conj()) # key scale moved to the query
    o_p, lse_p = flash_attn(qc, K, V) # prompt part
    o_p = o_p @ W.T # attention is linear in V
    o_a, lse_a = flash_attn(q, *kv_answer, causal=True)
    g = torch.sigmoid(lse_p - lse_a) # exact softmax over both
    return g * o_p + (1 - g) * o_a

def decode(token, K, V, m, n): # last prompt token, then answer
    h = embed(token)
    for k in range(n):
        j = max(m - n + k, 0) # aligned; one cache: j = m - 1
        for l in range(L):
            h = blocks[l](h, attn=lambda q, kv: attend(
                    q, K[j][l], V[j][l], kv, c[k][l], W[k][l]))
    return lm_head(h)
```

```
# training: one (m, n) per micro-batch,
# teacher = the Original 4/4 model
m, n = torch.randint(1, 5, (2,)).tolist()
with torch.no_grad():
    t = original(x, loops=4) # frozen, no adapter
s = model(x, prefill=m, decode=n) # LoRA + maps
kl = kl_div(s.log_softmax(-1),
          t.log_softmax(-1), log_target=True,
          reduction="none").sum(-1)
loss = kl[answer].sum() / (answer.sum() - len(x))
```

Figure 13: **LoopBridge in PyTorch-style pseudocode.** The fixed-depth adapter is the same code with $c = 1$, $W = I$ and m fixed. Decode loop k reads prefill loop $m - n + k$ (loops counted from zero here; the one-cache variant keeps only the last prefill loop) through its own key scale (applied to the query) and value map (applied to the attention output), and the prompt and answer attention are combined exactly through their log-sum-exp.

parts are computed by two attention calls with outputs o_P, o_A and log-sum-exps λ_P, λ_A of their attention logits, and merged as $g\,o_P + (1-g)\,o_A$ with $g = \sigma(\lambda_P - \lambda_A)$, which equals one softmax over both (the blockwise softmax of FlashAttention (Dao, 2023)). In the one-cache variant the prompt KV is therefore stored once for every n, and total KV is $b(\ell{+}n\,t)$ instead of $b\,n(\ell{+}t)$. Serving from one cache equals materialising a mapped prompt cache per loop; in bfloat16 all 512 checked greedy tokens agree.

Aligned caches (Ours). The main method keeps one prompt cache per decode loop and lets decode loop k read prefill loop $\max(m{-}n{+}k, 1)$ through its own map, so at $m = n$ every decode loop reads exactly the KV the original model would. It stores the prompt n times, as Original n/n does. The one-cache variant, trained identically, stores it

once and is slightly less accurate, most of all on long-context retrieval.

What the maps do. For the aligned adapter, the same training with the maps frozen at the identity moves ten of the 11 benchmarks by at most 2.8 points at 4/1 but lowers RULER-16K from 64.7 to 49.8 (Table 1). The one-cache variant shows the same pattern: with frozen maps it is within 0.4 points on MMLU (significantly lower at 1/1 and 2/2, by 0.4 and 0.3), has no significantly negative GSM8K difference (at 4/1 it scores 2.3 points higher $_{[0.2,\ 4.5]}$), but loses 11.4 points on RULER-16K at 4/1 (Appendix L). The maps matter for long prompts.

Baseline learning rate. The distilled controls use the adapter's learning-rate schedule. Scaling the whole schedule by 0.3 or 3 gives 11-benchmark averages of 49.2 and 50.1 for Distilled 1/1 (49.8 at the default) and 64.7 and 65.2 for Distilled 2/2 (65.3); at rank 80 with the threefold rate they are 50.0 and 65.6 (50.2 and 65.4 at the default). None reaches Ours at 4/1 or 4/2 (56.8, 67.2). The adapter trained the same way at 0.3 and 3 times its learning rate averages 55.5 and 57.4 at 4/1 and 66.8 and 67.8 at 4/2: even its weakest setting is above the best distilled control, and both methods are reported at the default rate.

Training. We distil (Hinton et al., 2015): we minimise $\mathrm{KL}(\text{teacher} \,\|\, \text{student})$ over answer positions, with the Original 4/4 model as teacher, on 48M answer tokens of reasoning traces, instruction data, long-context data and web text. One (m, n) pair is sampled per micro-batch, so one set of parameters (63M, 2.4% of Ouro-2.6B) serves all 16 pairs. Details are in Appendix G.

F Extended Related Work

Looped and elastic depth. Weight-tied depth recurrence goes back to Universal Transformers and cross-layer sharing (Dehghani et al., 2019; Lan et al., 2020); looped transformers are computationally universal (Giannou et al., 2023), can simulate chain-of-thought steps in latent space (Saunshi et al., 2025) and generalise in length with adaptive step counts (Fan et al., 2025b); continuous latent reasoning (Hao et al., 2025) instead feeds hidden states back as input tokens. Relaxed Recursive Transformers (Bae et al., 2025a) convert pretrained models into looped ones with per-loop LoRA; Mixture-of-Recursions (Bae et al., 2025b) and Huginn (Geiping et al., 2025) route or scale the loop count. Accordion (Zhang et al., 2026) and Elastic Latent Recurrence (Ladhak et al., 2026) train one model for several depth budgets; ELR finds that keeping only the last loop's KV retains most of the gain, as Dory1.0-4B (Fishman et al., 2026) does at any loop count. The Parallel Loop Transformer (Wu et al., 2025a), LoopCoder-v2 (Yang et al., 2026) and shared-memory looped transformers (Monea et al., 2026) let later loops read the first loop's KV plus a short local window. All run the same number of loops for prefill and decode. Ouro (Zhu et al., 2025) reuses the last loop's KV for generated tokens but loses more than 10 GSM8K points when prefill does the same; our adapter is trained to read prompt KV across loops. FlashLoop (Yang and Liu, 2026) speeds up looped models without training through token-sparse loop updates, sparse attention and KV-residual quantisation.

Adaptive and phase-specific computation. Halting (Graves, 2016; Banino et al., 2021), token-level early exit (Xin et al., 2020; Elbayad et al., 2020; Schuster et al., 2022; Elhoushi et al., 2024; Corro et al., 2023) and token routing (Raposo et al., 2024) spend depth per token; CALM must also fill the skipped layers' KV for later tokens, the problem our maps solve per loop. Speculative decoding (Leviathan et al., 2023; Chen et al., 2023) pairs a cheap drafter with a deep verifier, and pause tokens add compute at prompt positions (Goyal et al., 2024). Deep-encoder/shallow-decoder models (Kasai et al., 2021; Bergner et al., 2024; S et al., 2024) spend more depth on the prompt across separate networks. BridgeKV (Behnam et al., 2026), concurrent with us, prefills with a large model, decodes with a small one of the same family and translates the prompt KV cache between them with a learned module while both models stay frozen; LoopBridge maps the cache between loops of one model instead. SwiftKV (Qiao et al., 2025) instead skips later layers for prompt tokens. Continuous Depth Batching (Schwethelm et al., 2026) prefills at full depth, exits early during decode and studies caches shared across depths. Loop-Duet (Du et al., 2026), concurrent with us, sets the two loop budgets of Huginn without training; Huginn was trained with random loop counts, whereas fixed-depth Ouro fails without training (Table 1). Position-Aware Depth Decay (Fan

et al., 2025a) skips more layers for later tokens; OverFill (Kim et al., 2025) decodes with a width-pruned model; Fixed Points Part II (Huang et al., 2026) distils a light prefill network; POP (He et al., 2026), SPEED (Oh et al., 2026) and SEED (Ma et al., 2026) prefill with less computation than they decode; Decode-Branch Transformers (Liu et al., 2026), PHD (Wu et al., 2025b) and State-Prediction Separation (Langford et al., 2026) add decode-side computation. Serving systems already run the two phases on separate hardware (Zhong et al., 2024; Patel et al., 2024; Qin et al., 2025) or co-schedule them (Agrawal et al., 2024) over paged KV memory (Kwon et al., 2023); untying the loops lets each phase also use its own depth. We vary prefill and decode loops independently within one loop adapter and measure what each buys.

Asymmetric KV producers and consumers. YOCO (Sun et al., 2024) has a self-decoder produce one global KV cache that a cross-decoder reads; YOCO-U (Sun et al., 2026) loops the self-decoder, keeping KV storage independent of the loop count while every token still runs the loops; KITE (Hu et al., 2026) expands a checkpoint into two towers so that bulk prefill runs only the KV-producing tower. These architectures are built for the split; we adapt an existing looped model after pretraining and choose the prefill loops m and decode loops n at inference.

Sharing and transferring state. KV is shared across heads (Shazeer, 2019; Ainslie et al., 2023) and layers (Brandon et al., 2024; Zuhri et al., 2025), and evicted or compressed per token (Xiao et al., 2024; Zhang et al., 2023; Li et al., 2024; Cai et al., 2024), which is orthogonal to and composable with our per-loop reads. MELT (Vendrell et al., 2026) post-trains Ouro to keep one gated KV cache per layer shared by all loops plus a small per-loop sliding-window cache, decoupling memory from depth; it does not separate prefill from decode loops or map which depths a task needs. Looped Latent Attention (Neill and Reid, 2026) compresses KV across loops; BeNice! (Prairie et al., 2026) and LSTM-style looped transformers (Kirsch, 2026) share it; PLatour (Shen et al., 2026) injects a larger model's prefill states into a smaller decoder; Cache-to-Cache (Fu et al., 2026), HeteroFold (Yun et al., 2026), Swarm Telepathy (Alabdulkareem et al., 2026) and CacheLink (Qiu et al., 2026) transfer KV between models, and BellowsKV (Zhang and Chen, 2026) shares one latent across layers. Like CacheLink, we train our maps through the receiver's output; within one model tokens, layers and positions align, so a RoPE-commuting scale and a value map suffice. For hybrid attention / linear-recurrence models (Yang et al., 2025; Dong et al., 2025), state-swap interventions (Afendulev et al., 2026) and LatentPort (Villani, 2026) study the two kinds of state.

G Training Details

Loop adapter. Rank-16 LoRA ($\alpha = 16$, no dropout) on the query, key, value and output projections, active only when decoding; per (decode loop, layer, KV head) a complex scale for each rotary frequency pair on keys and a full $D \times D$ value map, both initialised to the identity. Trainable parameters: 63,307,776 for Ouro-2.6B. The backbone is frozen.

Objective and data. $\mathrm{KL}(\text{teacher} \,\|\, \text{student})$ summed over answer positions and normalised by $\sum_i (|a_i| - 1)$ over the answers a_i of the global batch (one less than the supervised positions per answer, as the released checkpoints were trained); teacher = Original 4/4 with the same weights; one pair (m, n) sampled per micro-batch. 48M answer tokens: OpenThoughts reasoning traces 45%, Tulu SFT responses 40%, LongAlpaca long-context answers 5% and FineWeb-Edu windows 10% (fractions of answer tokens); prompts up to 7,680 tokens, answers up to 1,024; the token budget counts $|a| - 1$ positions per answer a for the loop adapter and the distilled controls alike. Base models see a plain "User: … Assistant:" format; thinking models see the same samples in their chat template. No benchmark data was deliberately included; we did not decontaminate the training corpora. An audit afterwards (`overlap_audit.py`) finds a 13-token span shared with the training data in 187 of 542 code prompts (HumanEval+ and MBPP+), 101 of 500 MATH-500 questions, 106 of 14,042 MMLU, 75 of 6,511 BBH, 24 of 1,400 MMLU-Pro and at most 3 items of each other audited benchmark (GSM8K, ARC-Challenge, HellaSwag, Winogrande, GPQA, IFEval). Items whose extracted question text is unavailable or shorter than 13 tokens cannot be assessed (2,696 MMLU, 296 HellaSwag, 254 BBH, 247 ARC-Challenge, 194 MMLU-Pro, at most 4 elsewhere) and are kept; RULER and HELMET are synthetic

or long-document and not audited. Without the overlapping items, Ours at prefill 4 / decode 1 stays significantly above Distilled 1/1 (Table 1) on MMLU (+14.3), MMLU-Pro (+18.2), BBH (+12.4), GSM8K (+4.6), HumanEval+ (+10.0 on the 70 remaining problems) and MBPP+ (+5.3), and at decode 2 above Distilled 2/2 on MMLU (+6.3), MMLU-Pro (+7.7) and BBH (+2.6); 4-shot MATH-500 keeps the opposite sign (Appendix H).

Optimisation. AdamW ($\beta = (0.9, 0.95)$, no weight decay), peak learning rate 10^{-4} with 2% linear warm-up and cosine decay to 10^{-5}, gradient clipping at 1.0, global batch 24 sequences on 4 B300 GPUs (6.75 hours for Ouro-2.6B), bfloat16 backbone.

Shared answer KV cache. Generated tokens keep one KV cache per layer shared by all decode loops; at inference the last decode loop commits each token, and earlier loops read that history plus the current token's KV computed live in their own loop (*self-live*[1], a layout also used by LSTM-style looped transformers (Kirsch, 2026)). For example, when loop 1 processes token 5, it attends to tokens 1–4 through the last loop's stored KV and to token 5 through its own, freshly computed KV. Training uses the same KL objective in this layout, in two passes: a no-grad pass in which each loop reads its own causal answer KV records the last loop's answer KV; a second pass reads, at every loop, the prompt KV cache, that recorded history strictly before the current position, and the current token's KV computed live on the diagonal. Training thus approximates, but does not exactly reproduce, inference, where the history is generated recursively in the shared cache. With prompt and answer each stored once, total KV is $b(\ell + t)$, that of a one-loop model; with 1K-token prompts and 2,048 generated tokens the largest batch that fits grows from 32 (Original 4/4) to 192 and decode-4 throughput from 219 to 476 tokens/s (759 vs. 586 at n=2), at a cost of 3.0 GSM8K points at 4/4 against per-loop answer caches (Appendix L, Other design variants). On Ouro-2.6B-Thinking, a loop adapter trained the same way changes full-set GSM8K at Ours 4/2 by $+0.5$ [−0.5, 1.6] against per-loop answer caches; at batch 1 its observed cost per generated token is similar (82.5 vs. 83.1 ms) but its answers are longer (1,166 vs. 741 tokens), so it takes $1.53\times$ [1.27, 1.84] the time per answer. Its benefit is memory and batch size, not batch-1 latency.

[1]Self-live was developed by the author in separate work on shared KV caches for looped models; here it is applied to the loop adapter's answer tokens.

H Evaluation Details

Protocol. Base models follow the base-model protocol of the Ouro models (Zhu et al., 2025, Appendix C.1), run through our runner so that every (m, n) pair is evaluated the same way: lm-evaluation-harness for MMLU (5-shot, log-likelihood, 14,042), MMLU-Pro (5-shot CoT, strict match; 30 per subject, 420), BBH (3-shot CoT, strict match; 60 per subtask, 1,620), ARC-Challenge (25-shot, 1,172), HellaSwag (10-shot, 10,042), Winogrande (5-shot, 1,267) and GSM8K (3-shot CoT, strict match, 1,319); MATH-500 with the 4-shot Minerva prompt, scored on the final answer (the last `\boxed{}`, else the Minerva closing sentence) with math-verify (the Ouro models' own 5-shot prompt and scorer are not released); HumanEval+ and MBPP+ with evalplus as it runs for a tokenizer with a chat template (the problem in evalplus's chat prompt, greedy, 768 new tokens, sanitized code, plus tests). With this protocol original Ouro-2.6B 4/4 scores within one point of the published numbers on MMLU, MMLU-Pro, BBH, ARC-Challenge and HellaSwag, 1.9 above on Winogrande and within 1.2 to 3.1 points on HumanEval(+) and MBPP(+) (Table 9). Two scores differ for a documented reason: GSM8K strict matching requires the exemplars' "The answer is" sentence and gives 69.4, while flexible extraction of the last number gives 86.7, bracketing the published 81.6; and MATH-500 with the public 4-shot Minerva prompt gives 53.6 against 90.9 with Ouro's unreleased 5-shot prompt and scorer. Every configuration in this paper is scored the same way, but a protocol can change the size of a difference, not only the level. Rescoring the same GSM8K answers with flexible extraction, the aligned adapter at prefill 4 / decode 1 gains more over LoRA-distilled 1/1 (+6.7 [+4.0, +9.3] instead of +4.5), while at decode 2 its gain over Original 2/2 shrinks from +3.4 [+1.2, +5.6] to +1.2 [−0.8, +3.3].

GPQA-Diamond (0-shot), IFEval (prompt-level strict), RULER (13 tasks × 40 rows per length) and HELMET (RAG, synthetic recall, re-ranking and ICL; 1,108 rows) are prepared and scored by our own scripts. Thinking models answer

benchmark	ours	reported	items
MMLU	74.6	74.6	14,042
MMLU-Pro	56.1	55.7	1,400
BBH	80.7	80.5	6,511
ARC-C	66.5	66.4	1,172
HellaSwag	79.5	79.7	10,042
Winogrande	77.7	75.9	1,267
GSM8K	69.4	81.6	1,319
GSM8K (flexible extraction)	86.7	–	1,319
MATH-500	53.6	90.9	500
HumanEval	75.6	78.7	164
HumanEval+	69.5	70.7	164
MBPP	77.5	80.4	378
MBPP+	64.6	66.6	378

Table 9: **Reproducing the original model.** Ouro-2.6B 4/4 under our evaluation and as reported by Zhu et al. (2025, Table 8); MMLU-Pro and BBH here on the full runs of the original model.

GSM8K and MATH-500 zero-shot in their chat template, answers in `\boxed{}` scored with math-verify, with budgets of 4,096 and 8,192 tokens, greedily rather than with the sampling and model-judged scoring of their original evaluation; their MMLU-Pro and BBH are as for base models and their code as above. On raw HELMET prompts thinking checkpoints can emit a newline and stop; for thinking models we therefore mask the stop tokens until an answer has a non-whitespace token, for every configuration. Intervals: paired bootstrap over items, 5,000 resamples, recomputed from released per-item scores by one script (Appendix O).

Scoring and prompts. lm-evaluation-harness's strict Minerva match for MATH-500 credits an answer only if it ends with the exemplars' sentence "Final Answer: The final answer is X. I hope it is correct." Original Ouro-1.4B 4/4 writes boxed answers that are mostly correct but closes them in other words, so it scores 4.4 under strict matching and 70.6 on the final answer; on Ouro-2.6B the decoder with one prefill loop and four decode loops writes longer solutions than the exemplars, ends with that sentence on 183 of 500 problems (429 with four prefill loops), and scores 18.6 under strict matching but 68.2 on the final answer (51.0 with four prefill loops). We therefore score the final answer; the prefill-1 advantage on 4-shot MATH-500 is in the answers, not in their format. Code is different: under an earlier lm-evaluation-harness prompt, prefilling base Ouro-2.6B with two or more loops dropped Python indentation in 35–63% of MBPP+ answers (under 3% with one), and that task scored only the first base assertion; with evalplus this effect is gone. Original throughput baselines: static per-loop caches with the same two-call flash attention (fastest), the standard dynamic cache (500 / 245 / 105 tokens/s at $n = 1/2/4$), and a preallocated contiguous cache with one attention call (267 / 130 / 57); all produce identical greedy tokens.

I Other Checkpoints

Table 10 repeats Table 1 on Ouro-1.4B, and Table 11 compares prefill counts at one decode loop within the same adapter on four Ouro checkpoints.

With fixed configurations (Table 10), LoopBridge at 4/1 averages 43.9 on Ouro-1.4B, against 34.5 for Original 1/1 and 40.0 for Original 1/1 distilled on the same data.

method	m/n	MMLU	MMLU-Pro	BBH	ARC-C	HellaSw.	Winogr.	GSM8K	MATH	HE+	MBPP+	RULER	Avg.
Original	1/1	42.0	19.1	19.9	38.6	55.3	56.0	17.2	**24.2**	34.1	41.3	31.6	34.5
Training-free	4/1	32.8	0.0	0.0	31.7	27.8	51.1	0.0	0.0	0.0	0.0	0.0	13.0
Distilled	1/1	45.0	21.2	**32.8**	46.4	58.8	61.2	21.0	23.0	37.8	42.9	**49.8**	40.0
Ours, LoopBridge	4/1	**64.0**	**36.7**	13.4	**56.3**	**63.3**	**63.1**	**27.0**	**24.2**	**39.6**	**47.6**	47.2	**43.9**
Original	2/2	62.2	39.5	50.8	54.6	**71.5**	66.5	49.8	55.8	54.9	**57.7**	50.6	55.8
Training-free	4/2	37.5	1.2	0.0	35.1	30.9	57.3	0.0	0.0	0.0	0.0	8.5	15.5
Distilled	2/2	62.5	36.0	**55.7**	55.0	70.5	**67.9**	50.2	**58.0**	56.7	55.8	**63.1**	57.4
Ours, LoopBridge	4/2	**67.9**	**43.8**	49.4	**58.7**	71.4	67.6	**55.0**	53.2	**62.8**	57.1	53.5	**58.2**
Original	4/4	67.6	45.7	69.5	60.1	74.4	70.6	62.2	70.6	65.8	60.9	73.9	65.6

Table 10: **Table 1 on Ouro-1.4B**, with its own adapters and controls (MMLU 20 questions per subject and 400 GSM8K problems, as in its task map); each column on the items all rows share.

model (same decoder, decode 1)	knowledge	long context	multi-step
Ouro-2.6B, prefill 4 vs. 1	MMLU $+15.2$ [14.3, 16.1]	RULER-16K $+8.6$ [4.3, 12.8]	GSM8K $+8.5$ [6.1, 10.9]
Ouro-1.4B, prefill 4 vs. 1	ARC-C $+10.8$ [8.4, 13.1]	RULER-16K $+1.4$ [-2.6, 5.4]	GSM8K $+5.0$ [2.7, 7.2]
Ouro-2.6B-Thinking, prefill 4 vs. 1	MMLU-Pro $+18.6$ [13.6, 23.8]	HELMET $+7.7$ [2.1, 13.6]	GSM8K $+6.6$ [4.2, 9.2]
Ouro-1.4B-Thinking, prefill 4 vs. 1	MMLU-Pro $+17.1$ [12.4, 22.4]	HELMET $+4.6$ [0.1, 8.6]	GSM8K $+13.6$ [10.5, 16.6]

Table 11: **Prefill loops with the decoder held fixed** across four Ouro checkpoints (paired, within one adapter). Set sizes: MMLU 14,042, ARC-C 1,172, MMLU-Pro 420, RULER 520, HELMET 1,108, GSM8K 1,319. HELMET is averaged as its authors do (mean per dataset, then per category, then over its four categories). Each column shows one benchmark of that kind per checkpoint; counterexamples are in Appendix N. Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals.

J Prefill 4 / Decode 2 in Numbers

Table 12 lists the scores and paired intervals behind Figure 11, and Table 13 compares prefill 4 / decode 2 with Original 2/2 distilled on the same data.

K End-to-End Time and Decode Throughput

Batched serving. We also serve the 400 timed requests of Figure 7 in batches (Figure 14, Table 15). Up to B requests decode in lockstep with continuous batching: a finished request leaves the batch at once, and the next one is prefilled alone, at batch 1, while the others wait. There is no chunked prefill, which favours configurations with a short prefill. Each request keeps its cache in its own region of one buffer per loop and layer, and decode attention reads only that request's tokens (flash attention with variable lengths). Decoding is the same for every configuration: the n-loop model on per-loop prompt caches, with LoRA merged into the weights and, for LoopBridge, the maps folded into the caches. Time per request is the total wall time divided by the 400 requests, averaged over two timing passes: single runs of one task vary by 10–30% between passes (the generated tokens are identical), as eager decoding is bound by host-side work on a shared machine. Batch 1 is in Figure 7.

Serving. Storing the prompt once also changes throughput (Figure 12; Table 17 in Appendix K). In a controlled decode benchmark (8K context, 32 repeated decode steps, largest batch that fits on one B300), one prompt KV cache decodes $0.97\times$ as fast as the fastest implementation of the original model we measured at one decode loop and 1.19–$1.62\times$ at two to four, because it admits larger batches as n grows; this baseline uses the same flash attention and is faster than the standard dynamic cache (Appendix H). With long answers (1K-token prompts, 2,048 generated tokens) one prompt KV cache decodes $0.97\times$ Original at $n{=}2$ and $1.16\times$ at $n{=}4$. The gain over Original 4/4 grows with the prompt, from $2.4\times$ at a 2K-token prompt to $3.3\times$ at 15K, where Ours 4/2 also outpaces Original 2/2 ($1.12\times$). Aligned caches decode at about the cost of Original n/n (314 against 342 tokens/s at $n = 2$, $2.5\times$ Original 4/4).

		score				adapter 4/2 minus		
benchmark	N	Original 2/2	adapter 2/2	adapter 4/2	Original 4/4	Original 2/2	adapter 2/2	Original 4/4
GSM8K	1,319	92.2	93.0	93.3	94.2	+1.1 [−0.2, 2.4]	+0.2 [−1.0, 1.4]	−1.0 [−2.0, 0.1]
MATH-500	500	84.8	84.8	85.0	89.8	+0.2 [−2.6, 3.0]	+0.2 [−2.8, 3.2]	−4.8 [−7.8, −2.0]
MMLU-Pro	420	43.1	41.9	50.0	49.0	+6.9 [3.1, 11.0]	+8.1 [4.0, 12.1]	+1.0 [−2.6, 4.5]
ARC-C	1,172	62.5	63.8	66.1	67.5	+3.6 [2.0, 5.3]	+2.3 [0.9, 3.7]	−1.4 [−2.6, −0.1]
BBH	1,620	66.6	71.3	73.5	71.7	+6.9 [4.6, 9.2]	+2.2 [0.1, 4.3]	+1.9 [−0.4, 4.1]
HumanEval+	164	68.9	70.7	74.4	73.8	+5.5 [−0.6, 11.6]	+3.7 [−2.4, 9.8]	+0.6 [−4.9, 6.1]
MBPP+	378	61.9	61.6	62.4	60.6	+0.5 [−2.9, 4.0]	+0.8 [−2.4, 4.0]	+1.9 [−0.5, 4.2]
HELMET	1,108	44.0	49.5	57.0	65.7	+13.0 [6.7, 19.5]	+7.5 [1.4, 13.8]	−8.7 [−13.3, −4.1]

Table 12: **Figure 11 in numbers.** Ouro-2.6B-Thinking; one LoopBridge (one cache) serves prefill 2 / decode 2 and prefill 4 / decode 2; N: items all four configurations share. Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals.

		score			distilled 2/2	prefill 4 / decode 2
benchmark	N	Original 2/2	distilled 2/2	prefill 4 / decode 2	minus Original 2/2	minus distilled 2/2
GSM8K	1,319	92.2	93.0	93.3	+0.8 [−0.4, 2.0]	+0.2 [−1.0, 1.4]
MATH-500	500	84.8	84.6	85.0	−0.2 [−3.2, 2.8]	+0.4 [−2.2, 3.0]
MMLU-Pro	420	43.1	44.0	50.0	+1.0 [−2.9, 4.5]	+6.0 [1.7, 10.2]
ARC-C	1,172	62.5	63.0	66.1	+0.4 [−0.7, 1.5]	+3.2 [1.5, 4.8]
BBH	1,620	66.6	71.4	73.5	+4.8 [2.8, 6.9]	+2.1 [−0.1, 4.3]
HumanEval+	164	68.9	69.5	74.4	+0.6 [−4.3, 5.5]	+4.9 [−0.6, 11.0]
MBPP+	378	61.9	61.6	62.4	−0.3 [−2.6, 2.1]	+0.8 [−2.6, 4.5]
HELMET	1,108	44.0	53.8	57.0	+9.8 [6.9, 12.9]	+3.2 [−2.7, 9.3]

Table 13: **Untying versus distillation** (Ouro-2.6B-Thinking). "Distilled 2/2": rank-16 LoRA on the original two-loop model, trained with the same data, teacher (Original 4/4) and number of tokens as the loop adapter; N: items the three share. Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals.

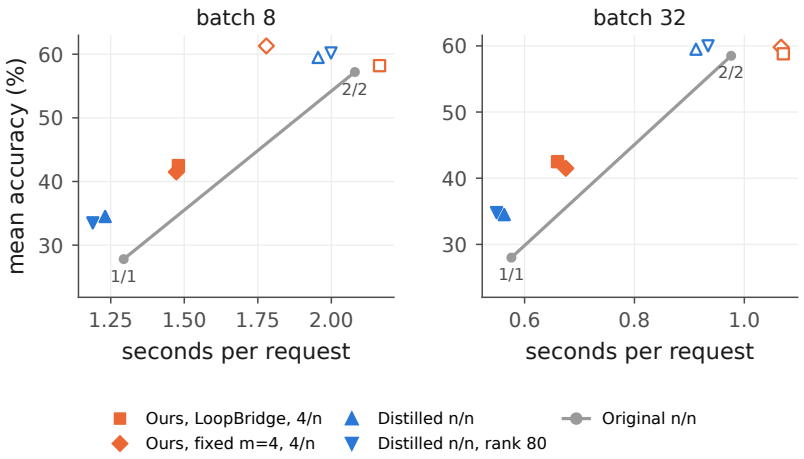


Figure 14: **Batched serving keeps the one-decode-loop advantage.** The 400 timed requests of Figure 7 served with continuous batching on one B300, each prompt prefilled alone: mean accuracy of the served answers against wall seconds per request (mean of two timing passes). Filled: one decode loop; hollow: two.

task / model	config	acc. (%)	s / answer	tokens	N
GSM8K Ouro-2.6B-T	Original 2/2	92.7	63.3	991	192
	Original 4/4	92.7	118	878	192
	distilled 2/2	94.3	57.7	837	192
	Ours 2/2	95.8	68.3	788	192
	Ours 4/2	94.3	69.6	788	192
MATH-500 Ouro-2.6B-T	Original 2/2	85.0	243	3191	40
	Original 4/4	82.5	469	3069	40
	Ours 4/2	85.0	298	3519	40

Table 14: **End to end per answer**, prefill included (batch 1, one B300; mean seconds and generated tokens per answer over N problems; GSM8K: consecutive problem IDs 1–192; budgets 4,096 and 8,192 tokens). On GSM8K, prefill 4 / decode 2 spends $1.38\times$ the decode time per generated token of Original 2/2 (pooled time over pooled tokens), but its answers are shorter.

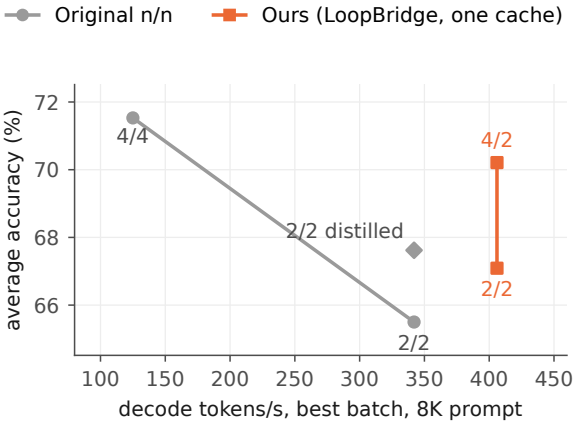


Figure 15: **Two decode loops come within 1.3 points of four on average at $3.2\times$ the decode throughput.** Ouro-2.6B-Thinking with LoopBridge (one cache): average accuracy over eight benchmarks (each on the items all configurations share) against a separate decode microbenchmark (best tokens/s over batch sizes at an 8K-token prompt, Table 17). Labels: prefill/decode loops; the four prefill loops add 3.1 points over 2/2 at the same decode cost.

configuration	acc.	s / request, batch 8	32	prefill at 32
Original 1/1	28.0	1.29	0.58	15%
Distilled 1/1	34.5	1.23	0.56	16%
Distilled 1/1, rank 80	34.8	1.19	0.55	15%
Ours, fixed m=4, 4/1	41.5	1.47	0.68	28%
Ours, LoopBridge, 4/1	42.5	1.48	0.66	27%
Original 2/2	58.5	2.08	0.98	15%
Distilled 2/2	59.5	1.96	0.91	16%
Distilled 2/2, rank 80	60.0	2.00	0.93	16%
Ours, fixed m=4, 4/2	59.8	1.78	1.07	22%
Ours, LoopBridge, 4/2	58.8	2.16	1.07	23%

Table 15: **Batched serving** (Figure 14): accuracy of the answers served at batch 32 (%), wall seconds per request, and the prefill share of the wall time at batch 32; ours shaded.

	GSM8K (400)			MATH-500	
	acc., b16	s, b16	s, b32	acc.	s, b16
Original 2/2	94.2	3.80	2.32	–	–
Original 4/4	95.5	6.82	3.86	–	–
Distilled 2/2	92.2	4.16	2.20	–	–
Ours, LoopBridge, 4/2	96.5	3.32	2.23	–	–

Table 16: **Batched serving, Ouro-2.6B-Thinking** (chat template; budgets 4,096 and 8,192 tokens): accuracy (%) and wall seconds per request at batch 16 and 32, mean of two timing passes; ours shaded.

8K context, 32 steps	n=1	n=2	n=3	n=4
Original n/n	834	342	200	125
Ours 4/n, one prompt cache	813	406	271	203
1K context, 2,048 steps		n=2		n=4
Original n/n		586		219
Ours 4/n, one prompt cache		570		253

Table 17: **Decode throughput** of Ouro-2.6B (tokens/s, best over tested batch sizes, one B300): original model against the loop adapter prefilling with 4 loops with one stored prompt KV cache.

L Design Variants

variant	m/n	task	difference
maps frozen at identity†	4/1	RULER-16K	-11.4 [-14.7, -8.2]
	4/1	GSM8K	$+2.3$ [0.2, 4.5]
	4/1	MMLU	-0.2 [-0.6, 0.1]
	4/2	RULER-16K	-2.6 [-4.5, -0.7]
	4/2	GSM8K	$+0.2$ [-1.3, 1.7]
unrestricted keys‡	2/2	RULER-16K	-3.4 [-5.6, -1.3]
	1/1	RULER-16K	$+0.1$ [-1.3, 1.5]
	2/2	RULER-16K	$+0.6$ [-0.8, 2.0]
	3/1	RULER-16K	-0.9 [-2.9, 1.2]
	3/2	RULER-16K	$+1.2$ [-0.4, 2.7]
	4/2	RULER-16K	-1.0 [-2.6, 0.5]

Table 18: **Design variants** on Ouro-2.6B: variant minus our loop adapter at prefill m / decode n (full sets, paired). Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals. †Same data, budget, seed and (m, n) sampling; only the maps are not trained (LoRA only, the decoder reads the last prefill loop's KV directly). ‡Unrestricted key maps with one prompt KV cache per decode loop.

Table 18 compares design variants. Training the same loop adapter with its maps frozen at the identity, so that a LoRA decoder reads the last prefill loop's KV directly, changes MMLU by at most 0.4 points and is not worse on GSM8K ($+2.3$ [0.2, 4.5] at prefill 4 / decode 1), but lowers RULER-16K by 11.4 points at prefill 4 / decode 1 and by 2.6–3.4 points at two decode loops. Restricting the key map to a RoPE-commuting scale, which is what allows one stored prompt KV cache, shows no significant difference against unrestricted key maps with one cache per loop in five tested configurations (RULER-16K; Table 18).

The maps beyond RULER. Table 19 compares the three all-pairs adapters of Section 5.6 on two more long-context benchmarks, HELMET (1,108 items, median prompt 9.4K tokens) and LongBench (400 items, median 10.5K tokens). The advantage of the maps is specific to exact retrieval over the longest prompts with one decode loop (RULER-16K at 4/1). On HELMET and LongBench, LoopBridge stays within 0.3 points of the rank-16 LoRA at both depths, while the rank-80 LoRA is 2.4 and 2.9 points above it on HELMET (LoopBridge minus rank 80: [-5.0, -0.3] at 4/1, [-6.2, 0.0] at 4/2) and on LongBench 1.2 points above it at 4/1 and 1.3 below it at 4/2. Over three training seeds of each, HELMET at 4/1 is 32.7 ± 1.2 for LoopBridge and 34.5 ± 1.1 for the rank-80 LoRA.

m/n	adapter	RULER-16K	HELMET	LongBench
4/1	LoRA r16	49.8	32.0	8.5
4/1	LoRA r80	55.8	34.7	9.8
4/1	LoopBridge	64.7	32.2	8.6
4/2	LoRA r16	71.6	48.6	13.6
4/2	LoRA r80	73.7	51.2	12.1
4/2	LoopBridge	73.5	48.3	13.4

Table 19: **The maps on three long-context benchmarks**, Ouro-2.6B, first training seed; one adapter for all (m, n) in every row; each benchmark on the items the three share, HELMET aggregated as its authors do.

Other design variants. Sharing generated-token KV across decode loops ("shared answer KV", Appendix G) costs 2.4 to 3.0 GSM8K points (-2.5 [-4.5, -0.5], -2.4 [-4.2, -0.7], -3.0 [-4.8, -1.3] at 4/2, 3/3, 4/4). A gated branch over the first prefill loop's KV improves RULER-4K by 2.3 [1.0, 3.6] points at prefill 4 / decode 1 with no significant difference on GSM8K ($+0.5$ [-0.9, 2.0]).

M Looped Decision Models: Setup and Checks

We evaluate on the 3,270 answerable questions of LevBench's public development set (never its locked test set), with intervals over record groups as Lev does. Each prefill count m is one continued training (seed 0); the main comparison (m equal to the backbone's loops) was repeated with three more seeds and against a control trained for twice as many updates. With all decision loops, the model with the most prefill loops is no better than the control on the Ouro backbones and below it on Lev-3B (-1.8 [-3.0, -0.7]); with $m = n$ the trained models are within a few points of the control in both directions (Table 20).

Setup and checks. Both arms continue each released Lev model for 140 updates on the 8,925-record public S31 training subset with Lev's hyperparameters, from the same weights. Our reference forward runs the full sequence through both the prefill and the decision loops, so its training computation is about twice the control's. Without training, more prefill loops do not help (-3.0 on Lev-1.4B at one decision loop), and the control itself moves little against the released models at one loop ($+0.5$ on Lev-1.4B, $+1.3$ on Lev-2.6B). Repeating both arms with other seeds, the one-loop gain ranges from $+3.9$ to $+5.3$ on Lev-1.4B and from $+5.1$ to $+7.5$ on Lev-2.6B, every interval excluding zero (four seeds each); on

model	prefill m	decision 1	decision 2	decision 3	decision 4
Lev-1.4B	1	$+2.5^{*}$ [1.3, 3.7]	-2.8^{*} [−4.2, −1.4]	-6.1^{*} [−7.8, −4.6]	-9.6^{*} [−11.2, −7.9]
Lev-1.4B	2	$+2.9^{*}$ [1.3, 4.7]	$+2.6^{*}$ [1.4, 3.8]	-0.1 [−1.3, 1.1]	-2.7^{*} [−4.0, −1.4]
Lev-1.4B	3	$+5.7^{*}$ [3.9, 7.6]	$+3.8^{*}$ [2.3, 5.1]	$+1.8^{*}$ [0.7, 3.0]	-1.2^{*} [−2.4, −0.03]
Lev-1.4B	4	$+5.3^{*}$ [3.4, 7.2]	$+3.8^{*}$ [2.3, 5.3]	$+2.0^{*}$ [0.7, 3.2]	-0.1 [−1.3, 1.1]
Lev-2.6B	1	$+0.7$ [−0.5, 2.0]	-7.5^{*} [−9.1, −5.9]	-12.5^{*} [−14.1, −10.8]	-13.0^{*} [−14.7, −11.3]
Lev-2.6B	2	$+4.4^{*}$ [2.6, 6.2]	-0.8 [−2.0, 0.4]	-3.5^{*} [−4.8, −2.3]	-4.1^{*} [−5.5, −2.8]
Lev-2.6B	3	$+6.8^{*}$ [4.8, 8.7]	$+2.3^{*}$ [0.9, 3.8]	-0.4 [−1.5, 0.7]	-0.9 [−2.0, 0.3]
Lev-2.6B	4	$+7.5^{*}$ [5.7, 9.5]	$+3.5^{*}$ [2.0, 5.0]	$+0.5$ [−0.7, 1.7]	-0.3 [−1.4, 0.8]
Lev-3B	1	$+0.3$ [−0.7, 1.3]	-8.1^{*} [−9.7, −6.6]	–	–
Lev-3B	2	$+4.4^{*}$ [2.8, 6.1]	-1.8^{*} [−3.0, −0.7]	–	–

Table 20: **Lev decision models, both directions.** Accuracy of the model trained to prefill the context with m loops minus the matched-data and matched-update control (Original n/n) at n decision loops, LevBench development set (3,270 answerable questions), in points, training seed 0 (other seeds in the text); brackets are 95% bootstrap intervals over record groups (*: interval excludes zero). Lev-3B's backbone has two loops. Above the diagonal ($m < n$) every difference is negative.

Lev-3B it is $+4.4$, $+5.1$, $+4.0$ and $+0.9$ [−0.8, 2.7] (mean $+3.6$). A control trained for two epochs (twice the updates, about the training computation of the other arm) still trails it at one decision loop by $+4.0$ [2.1, 6.0] on Lev-1.4B, $+5.5$ [3.7, 7.4] on Lev-2.6B and $+4.0$ [2.3, 5.7] on Lev-3B.

N Prompt Format and HELMET Categories

The prompt format decides MATH-500. With 4-shot exemplars, one prefill loop is best for Loop-Bridge: 34.8 against 25.4 for four at one decode loop, and 63.8 against 45.8 at two. Without the exemplars four prefill loops are better, by 8.0 [4.0, 11.8] points at one decode loop and 10.4 [6.6, 14.2] at two.

HELMET shows which kinds of long-context task depend on the prefill loops (Table 21). On both Ouro reasoning models, prefilling with four loops instead of one improves retrieval-augmented QA by 15 points and re-ranking by 13–18, but lowers exact recall from a JSON key–value store on Ouro-1.4B-Thinking (-23.5). The recall category is small (34 items).

comparison	RAG	re-rank	ICL	recall
2.6B-T, prefill $4-1$	$+15.4$	$+13.4$	$+5.1$	-2.9
1.4B-T, prefill $4-1$	$+14.5$	$+18.0$	$+9.3$	-23.5

Table 21: **HELMET by category** (score differences on a 0–100 scale; prefill comparisons at one decode loop, same loop adapter). More prefill loops help retrieval-augmented QA (RAG), re-ranking and in-context learning (ICL) on Ouro but can lose exact key–value recall. Items: 700 / 41 / 333 / 34 per category; intervals are in `paper_numbers.json`.

The effect of the prefill loops depends on the prompt format (Table 22). At one decode loop, more prefill loops give a negative but inconclusive estimate on RULER-4K, where answers are copied verbatim, and lower 4-shot MATH-500 on both model sizes; without exemplars the same comparison turns positive on two model sizes and two decode loop counts. The aligned adapter (Ours) behaves the same way: four prefill loops instead of one change 4-shot MATH-500 by -9.4 [−13.6, −5.4] and -18.0 [−22.2, −14.0] points at one and two decode loops, and 0-shot MATH-500 by $+8.0$ and $+10.4$. Removing exemplars also changes length and format, so we read this as an interaction with the prompt.

model	decode n	task	prefill $4-$ prefill n
Ouro-2.6B	1	RULER-4K	-3.8 [−8.2, 0.3]
	1	MATH, 4-shot	-12.0 [−16.2, −7.8]
	1	MATH, 0-shot	$+8.0$ [4.2, 11.6]
	2	MATH, 0-shot	$+7.4$ [3.4, 11.4]
Ouro-1.4B	1	MATH, 4-shot	-4.4 [−8.4, −0.6]
	1	MATH, 0-shot	$+4.8$ [1.2, 8.4]

Table 22: **The effect of prefill loops changes with the prompt setup.** Same loop adapter, base models, prefill 4 / decode n minus prefill n / decode n; MATH-500 scored on the final answer. Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals.

O Reproducibility

`python ouro/scripts/paper_numbers.py compute` recomputes the paired differences, intervals, means, timing ratios and decode throughput of this paper from the accompanying per-item records and compares them with expected values hard-coded in the script, which mirror the numbers

printed here; only the MBPP+ format counts of Appendix H are reported as measured. Every run's command, configuration, checkpoint hash, log and outputs are in `ouro/results/provenance`. Re-running the main-table cells from scratch reproduces GSM8K and MMLU with at least 99.6% of item scores identical and RULER-16K with 94–100% identical and averages within 0.9 points (`ouro/results/verify_rerun.txt`). `REPRODUCE.md` maps every result to its scripts and gives the training and evaluation commands that regenerate the per-item records.